Comparing Statistical Learning Models for Reinforcement Learning

Ioan Baker

Master of Mathematics with Honours
School of Mathematics
University of Edinburgh
2026

Abstract

We investigate the interpretation of Stein Variational Policy Gradient (SVPG) as a variational inference method for approximating a Bayesian posterior over policy parameters. In particular, we study the effect of prior choice and temperature on the dynamics of particle learning. We find that even weakly informative priors can have a strong and persistent influence on learning. By examining the magnitudes of the competing policy gradient and prior gradient terms throughout training, we show that the policy gradient does not behave like a standard likelihood that becomes increasingly sharply peaked as more data is collected, hence the contribution of the prior does not diminish with training. In fact, while standard Bayesian inference expects the likelihood gradient to scale with the number of data points, the policy gradient approaches zero near a local optimum, resulting in the prior gradient dominating learning. This weakens the ordinary Bayesian interpretation of SVPG, and we see that the formulation is more similar to general Bayesian inference, in which a loss function replaces the negative log-likelihood. We show that, with genuine prior information, using an informative prior centred on the location of a high-return region can substantially improve learning with greater stability than simply using this information to initialise the particles. In this case, the constant attraction towards the mass of the prior helps to stabilise learning as particles are free to explore a controlled region around the predetermined area of high return.

Declaration

I declare that this thesis was composed by myself and that the work contained therein is my own, except where explicitly stated otherwise in the text.

I declare that ChatGPT was used in the completion of this work for code debugging and enhancing the appearance of prior predictive check plots in section 5.1.

(*Ioan Baker*)

Contents

Abstract ii

1 Introduction 1

2 Reinforcement Learning Foundations 3
- 2.1 The Reinforcement Learning Problem . . . 3
  - 2.1.1 The Environment . . . 3
  - 2.1.2 Action Selection . . . 6
  - 2.1.3 Value and Optimality . . . 7
- 2.2 Tabular Methods . . . 9
  - 2.2.1 Monte Carlo and Temporal Difference Learning . . . 9

3 Deep Reinforcement Learning 12
- 3.1 Value-based vs Policy Gradient Methods . . . 12
- 3.2 Stochastic vs Deterministic Policies . . . 13
  - 3.2.1 Policy Representation . . . 14
- 3.3 Policy Gradient Theorem and REINFORCE . . . 15
  - 3.3.1 Variance Reduction . . . 17
  - 3.3.2 Computational Analysis . . . 19
  - 3.3.3 Summary . . . 23
- 3.4 Actor-Critic Methods . . . 23
  - 3.4.1 Generalised Advantage Estimation . . . 25
- 3.5 Conclusion . . . 26

4 Bayesian Inference for Reinforcement Learning 28
- 4.1 Uncertainty in Policy Gradient Methods . . . 28
- 4.2 Bayesian Inference . . . 29
  - 4.2.1 Prior Distribution . . . 30
- 4.3 Variational Inference . . . 31
  - 4.3.1 Stein Variational Gradient Descent . . . 32
  - 4.3.2 Stein Variational Policy Gradient . . . 33

5 Experiments 35
- 5.1 Prior Predictive Check . . . 36
  - 5.1.1 Results . . . 39
- 5.2 Prior Performance . . . 40
  - 5.2.1 MAP estimation . . . 44

5.3 Genuine Prior Information 47

6 Results and Discussion **50**
6.1 Results 50
6.1.1 Limitations of the Experiments 51
6.2 Issue with the Bayesian Formulation 52
6.2.1 Finite-Particle and Gradient Approximation 52
6.2.2 Absence of a True Fixed Likelihood 53
6.2.3 Persistence of the Prior 54
6.3 Alternative Interpretations 54
6.3.1 Boltzmann/Gibbs Distribution 54
6.3.2 General Bayesian Inference 55

7 Conclusion **56**

References **58**

Chapter 1

Introduction

Reinforcement Learning (RL) is a branch of machine learning that can be thought of as the art of learning through trial and error. Unlike supervised and unsupervised learning, RL does not learn from a fixed dataset. Instead, an agent must discover, through experience interacting with a highly complex environment, which strategy, or policy, leads to the most desirable outcome. This formulation requires relatively little prior knowledge about the structure of the problem, and modern approaches have been shown to produce highly effective policies with sparse information. One of the most fundamental principles in RL is the reward hypothesis posited by Sutton and Barto (2018) and later formalised by Bowling et al. (2023), which states "all of what we mean by goals and purposes can be well thought of as maximisation of the expected value of the cumulative sum of a received scalar signal". The lack of required prior information, combined with the reward hypothesis, makes RL a remarkably general framework for machine learning.

Over the past few decades, RL has been used in a vast array of practical applications. A few of these include producing superhuman-level performance in games such as Atari, Go and chess (Mnih, 2013; Silver et al., 2017), self-driving cars (Djerbi et al., 2025), robotics (Kober et al., 2013), mimicking animal behaviour (Yamaguchi et al., 2018), medication administration (Raghu et al., 2017; Huang et al., 2022; Nemati et al., 2016) and large language models (Havrilla et al., 2024).

However, many current algorithms aim to produce a single point estimate of the optimal strategy despite the numerous sources of uncertainty and error that arise during training. While these methods have demonstrated high performance, there is no clear way to understand how the predicted optimal strategies are distributed. This means we have little insight into how uncertain or sensitive the resulting policy is to environmental changes. For high-risk applications, such as medication administration, understanding this resulting uncertainty is vital to avoid fatal consequences.

Because of this, the field of Bayesian RL aims to infer posterior distributions rather than point estimates. This distribution can then be used to quantify the uncertainty in our prediction, allowing us to make more informed decisions in critical situations. The complexity of RL problems prevents the use of exact Bayesian inference, and instead, we focus on approximate inference methods such

as variational inference, which this report addresses.

This dissertation investigates uncertainty in policy gradient methods and how Bayesian reinforcement learning attempts to address this, as well as other forms of uncertainty. In particular, we study Stein Variational Policy Gradient (SVPG) (Liu et al., 2017), a variational inference method that maintains a distribution over policy parameters by training a set of interacting particles representing unique policies. Our primary focus is whether SVPG can be interpreted as approximating a Bayesian posterior over policy parameters, and how prior information influences both learning dynamics and the Bayesian interpretation. We implement the algorithm and study the effects of prior choice and temperature on learning, and investigate the consequences of using informative priors.

This report begins by providing a crucial outline of the theoretical foundations of reinforcement learning, including the formal definition of the environment, the goal of reinforcement learning, and how value functions are defined. We begin as generally as possible before narrowing the discussion towards the algorithm we later investigate. We also briefly cover tabular methods such as dynamic programming, a method of optimal control when environment dynamics are known, as well as temporal difference learning and Monte Carlo methods.

This leads to a discussion of deep reinforcement learning for high-dimensional or continuous control problems, and in particular, methods of estimating the policy gradient. In addition to providing a theoretical overview, we perform an empirical analysis of the variance of several gradient estimators produced by these methods. We demonstrate the effect that this has on learning by training different agents to solve a standard control problem. Understanding the challenge of estimating the policy gradient is vital, as this term plays a key role in the SVPG posterior formulation.

We then examine a Bayesian approach to reinforcement learning by considering the various forms of uncertainty in the RL process. In particular, we study variational inference methods such as Stein Variational Gradient Descent (SVGD) and the resulting policy gradient algorithm, Stein Variational Policy Gradient.

The final chapters then examine SVPG, both experimentally and theoretically. We first perform prior predictive checks to understand the behaviour induced by different prior distributions over policy parameters. Next, we study how prior choice and temperature affect learning performance, and examine the contributions of the policy and prior gradient terms during training. Finally, we investigate the case of genuine prior information to determine the effect on learning if we incorporate this prior information at initialisation or in the SVPG update. These experiments allow us to identify several issues with interpreting SVPG as an approximation to an ordinary Bayesian posterior. One critique is that the policy-gradient term does not behave like a standard likelihood function for a number of reasons, and we therefore discuss alternative interpretations for the SVPG target. Namely, the Boltzmann distribution and general Bayesian inference.

Chapter 2

Reinforcement Learning Foundations

In this chapter, we will begin by formally defining the general RL problem and the ways in which a given problem can be categorised. This will begin as a general formulation before narrowing our discussion to the class of problems suitable for Stein Variational Policy Gradient (Liu et al., 2017), which will be covered in Chapter 4. We will then provide a brief description of the Bellman equations and tabular methods that are used to solve the simplest examples. Each of these algorithms has serious limitations or specific requirements that prevent them from being used for more complex, practically applicable problems. However, various aspects of their methodology are important to understand to build a foundation for future chapters.

2.1 The Reinforcement Learning Problem

Every RL problem consists of one or more **agents** which act in an **environment** at time steps t (Sutton and Barto, 2018). These time steps may be continuous, as is often the case for physical systems such as in the field of robotics (Kober et al., 2013; Doya, 1996). However, due to the inevitable discretisation of time caused by computer simulation, typically t is assumed to be discrete and nonnegative, $t \in \mathbb{N}$, as in this report. The agent observes information about the environment at this timestep s_t, selects an action using this information a_t, and then performs the action, resulting in an observed scalar reward r_t before observing some new information. The standard goal is to determine a **policy** for selecting actions based on the observed information which maximises the long-term expected discounted rewards it receives. The environment is the system that determines the way in which the agent's actions affect future states as well as the rewards that it receives.

2.1.1 The Environment

For this chapter, we will refer to the Frozen Lake environment (Farama Foundation, 2025*a*) shown in Figure 2.1 to frame specific examples. This is a simple en-

vironment in which the agent (top-left) must navigate across discrete grid squares to reach the goal (bottom-right) in the shortest amount of time while avoiding the icy ponds.

Figure 2.1: Example stochastic gridworld environment from Farama Foundation (2025*a*)

The environment is most commonly modelled as a Markov decision process (MDP), though non-Markovian environments can also be used, (Chandak et al., 2024; Whitehead and Lin, 1995; Gaon and Brafman, 2020). The general MDP is represented as a 5-tuple

$$(\mathcal{S}, \mathcal{A}, P_t, R_t, \rho_0),$$

where $\mathcal{S}, \mathcal{A}$ are the set of possible states the agent can be in and the set of actions they may take, respectively. We denote the state and action random variables at time t as S_t and A_t respectively, while realisations of these random variables are denoted as lowercase s_t and a_t. These sets may each be infinite and may also each be continuous. For our example environment, $\mathcal{S}$ is the discrete set of each of the 16 grid squares and $\mathcal{A} = \{\text{up, down, left, right}\}$ is the discrete set of actions.

The **transition function** (or environment model),

$$P_t(s'|s, a) := Pr(S_{t+1} = s'|S_t = s, A_t = a),$$

determines the dynamics of the environment by returning a probability of moving from state $s \in \mathcal{S}$ at time t to each state $s' \in \mathcal{S}$ given the current action $a \in \mathcal{A}$. This gives us the distribution of future states $S_{t+1} \sim P_t(\cdot|s_t, a_t)$. **Deterministic transition functions** satisfy $P_t(s'|s, a) = 1$ for some $s' \in \mathcal{S}$ and $P_t(s^*|s, a) = 0$ for all other $s^* \in \mathcal{S}$ for every $t \in \mathbb{N}$. In the Frozen Lake environment, we can model the transition function as a deterministic mapping, moving the agent in the direction of the action that it selects. We could also model a stochastic transition function by incorporating slipping and moving the agent one square in the chosen direction with probability 0.9 and two squares with probability 0.1. Stochastic transition functions represent a form of randomness that we cannot remove, called aleatoric uncertainty, which will be covered in Chapter 4.

The agent most often lacks explicit knowledge of the environment model to use during training and uses it only implicitly through interactions with the environment. Such problems are referred to as **model-free** and are the focus of this

dissertation due to their greater applicability to real-life problems where exact transitions are often overwhelmingly complex and unknown. In the case that the transition function is given to the agent, a set of optimisation algorithms known as dynamic programming (Bellman, 1954) can be used to converge to the exact optimal policy. This is an example of **optimal control** covered briefly in Section 2.2.

We next have the random variable $R_t(s, a)$ which represents the scalar reward observed by the agent when it takes action a from state s at time t. The realised reward obtained is r_t so that if $S_t = s_t$ and $A_t = a_t$, then r_t is a scalar realisation of the random variable $R_t(s_t, a_t)$. In many environments, this reward distribution is deterministic, in which case the reward function $R_t(s, a)$ represents the exact scalar reward. For ease of notation, we will now make the standard assumption that rewards are deterministic unless stated otherwise. Larger rewards represent more desirable events, while negative rewards may be used to deter the agent from taking the corresponding state action pair. In the Frozen Lake environment, we would like to reward the agent with a large value for reaching the final goal and penalise them for falling in a lake. This penalty may be a large negative reward or even just terminating the episode, preventing the agent from reaching the final goal. If we would like them to reach the goal in a short amount of time, we might also incorporate a small, constant negative reward at each timestep. Tuning the relative weight of these rewards and introducing additional rewards to elicit more nuanced behaviours is the challenge of **reward shaping**. For instance, the game of chess has a clear winning and losing state, so we could assign a $+1$ if the agent achieves a checkmate and wins, and a -1 if it loses. While acceptable, using such sparse rewards can make learning incredibly challenging since the agent has very little information telling it what actions led to the final outcome. This is known as the **credit assignment problem** and will be explored later in this report, where we discuss the ways that different algorithms attempt to solve this problem by estimating value functions and future rewards. The difficulty of designing appropriate reward functions motivates inverse reinforcement learning (Arora and Doshi, 2021; Ng et al., 2000), a field that attempts to learn the reward function that a given expert policy is implicitly optimising. One possible application of inverse reinforcement learning is the simulation of animal behaviour, as in Yamaguchi et al. (2018), by observing actions taken by the animal and attempting to infer the states that it regards as having higher value.

All MDPs satisfy the **Markov property**, meaning the transition and reward functions are memoryless and depend only on the current state and action. Therefore, $P_t(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = P_t(s_{t+1}|s_t, a_t)$ and analogously with the reward function. Non-Markovian environments do not satisfy this and lead to much more complex learning problems. For example, an environment in which entering a certain state produces a reward only if it has not been visited before is non-Markovian if the state only represents the agent's location. These problems can often be converted into MDPs by expanding the state space with indicator variables such as state visited $\in \{0, 1\}$. Sometimes this enlargement can result in an infinite or infeasibly large state space, which is why it is important to study the case of non-Markovian decision processes separately.

The process is **stationary** if $P_t(s'|s, a)$ and $R_t(s, a)$ do not change with time

such that $P_t(s'|s,a) = P(s'|s,a)$ and $R_t(s,a) = R(s,a)$ for all $t \in \mathbb{N}$. Non-stationary MDPs violate this, meaning the reward and transition functions can experience gradual changes or sudden jumps. In Figure 2.1, these problems could represent a new lake or a new route appearing in the grid, causing a sudden change in the reward or transition function. Alternatively, the probability of slipping increasing over time is an example of a less sudden non-stationary transition function. The common example of a moving bandit is covered by Sutton and Barto (2018), and often, non-stationary MDPs are approached by ensuring adequate and well-maintained exploration throughout training and deployment.

Finally, ρ_0 is the **initial state distribution**, from which the agent's starting state is sampled so that $s_0 \sim \rho_0$. A large number of environments, such as CartPole (Farama Foundation, 2024*a*), use a deterministic transition and reward function, meaning that for a given method of action selection, the only randomness from the environment comes from the initial state distribution.

2.1.2 Action Selection

At each time step, the agent selects actions according to a **policy** $\pi_t(a|s)$. Similarly to the transition function, stochastic policies represent a probability distribution over actions for each input state, while deterministic policies return a single action $a = \pi_t(s)$. We also define stationary policies which satisfy $\pi_t(a|s) = \pi(a|s)$ for all $t \in \mathbb{N}$. Standard reinforcement learning involves an agent in a state $s_0 \sim \rho_0$ taking an action $a_0 \in \mathcal{A}$ dictated by the policy $a_0 \sim \pi_0(\cdot|s_0)$, entering a new state $s_1 \sim P_0(\cdot|s_0, a_0)$[1] determined by the transition function and observing a reward $r_0 = R_0(s_0, a_0)$ from the reward function.

For environments with a finite horizon T' where the process is truncated, this action selection and execution is repeated until $t = T \leq T'$. Truncation represents a fixed time limit, and T may be less than T' if the environment is terminated early, for example, when the agent reaches the final goal or falls into a lake. At time $t = T$, the **episode** ends and we define the resulting **trajectory**

$$\tau := (s_0, a_0, r_0, s_1, a_1, r_1, \ldots, s_{T-1}, a_{T-1}, r_{T-1}, s_T).$$

It is also possible to have environments with infinite time horizons, which could be the case for Figure 2.1 if we do not incorporate a timestep limit and the agent moves indefinitely between two squares (and if the transitions between these squares are deterministic). In this case, we can never collect a full trajectory and must use alternative learning approaches such as bootstrapping or truncation discussed in Chapter 3. The distribution of states and actions in trajectories induced by policy π is

$$\rho_\pi(\tau) := \rho_0(s_0) \prod_{t=0}^{T-1} \pi_t(a_t|s_t) P_t(s_{t+1}|s_t, a_t). \tag{2.1}$$

[1]For fully observable MDPs, the agent will always have access to observations of its current state. However, partially observable MDPs (POMDPs) (Littman, 2009) are the more general case where the agent receives information generated from an observation function $O(z|s', a)$ relating states and actions to the agent's observations.

This shows that randomness in trajectories arises from the stochastic policy, transition dynamics, and initial state distribution. Additional randomness may arise from a stochastic reward function, but this does not affect the states visited and actions taken.

As mentioned at the beginning of the chapter, the goal of standard reinforcement learning is to find the policy $\pi_t(a|s)$ which, when followed, leads to the maximum expected long-term discounted reward. The long-term reward from time t is $\sum_{i=0}^{T-t-1} r_{t+i}$. These rewards are almost always discounted over time, both to ensure convergence of the sum in the case $T \to \infty$, as well as to account for the innate preference towards more immediate rewards. A **discount factor** $\gamma \in [0, 1]$, sometimes included in the definition of the MDP, is used to give us the long-term discounted rewards from time t [2]

$$G_t = \sum_{i=0}^{T-t-1} \gamma^i r_{t+i}, \tag{2.2}$$

referred to as the **return** from time t. For infinite horizons, if the rewards are bounded by $|r_t| \leq r_{\max}$ and $\gamma < 1$, then we see that

$$\left| \sum_{i=0}^{\infty} \gamma^i r_{t+i} \right| \leq r_{\max} \sum_{i=0}^{\infty} \gamma^i = \frac{r_{\max}}{1-\gamma} < \infty,$$

hence the return is bounded.

Unless mentioned otherwise, in the remainder of this report, when we speak of an RL problem, it is assumed that the problem consists of a single agent acting within discrete time steps using a stationary policy in a fully observable and stationary MDP. This is the formulation that is required for our experiments in Chapter 5.

2.1.3 Value and Optimality

In order to produce a method for learning an optimal policy, we must first define a set of value functions that represent the expected return under specific conditions. Firstly, the expected return of the agent under policy π is denoted

$$J(\pi) := \mathbb{E}_{\tau \sim \rho_\pi}[G_0] = \mathbb{E}_{\tau \sim \rho_\pi}[\sum_{i=0}^{T-1} \gamma^i r_i]. \tag{2.3}$$

This is the objective function that we seek to maximise and plays a key role in policy gradient methods on which this report focuses.

Since the objective function only describes the initial expected return, we would like to define a way to identify which individual states have high value, as well as to determine the value of taking certain actions in those states. The

[2] We define the most myopic discounted returns using the discount factor $\gamma = 0$ as $G_t = r_t$, although this is rarely useful. Typically, γ is chosen to be well above 0.5 and often close to 1 so that we can use reward information from distant time steps.

state value function

$$V^\pi(s) := \mathbb{E}_{\tau\sim\rho_\pi}[G_t|S_t = s]$$

is the expected return *from the state* s while acting with policy π. Finally, the **action value function**

$$Q^\pi(s,a) := \mathbb{E}_{\tau\sim\rho_\pi}[G_t|S_t = s, A_t = a]$$

is the expected return of the agent from state s *if they first take action* a and act according to π thereafter. We can see immediately that $J(\pi) = \mathbb{E}_{s_0\sim\rho_0}[V^\pi(s_0)]$ giving us an alternative view of our objective. There are several ways that we can link these value functions, and doing so in a recursive manner leads us to the **Bellman equations** derived below.

Firstly, we can expand the action value function after the first action is executed and use the corresponding state value function to see

$$Q^\pi(s,a) = R(s,a) + \gamma\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[V^\pi(s')\right].$$

Combining this by writing the state value function written as an expectation over all actions of the action value function,

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[Q^\pi(s,a)\right], \tag{2.4}$$

we arrive at the recursive Bellman equation for the state value function

$$V^\pi(s) = \mathbb{E}_{a\sim\pi(\cdot|s)}\left[R(s,a) + \gamma\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[V^\pi(s')\right]\right]. \tag{2.5}$$

Similarly, we can substitute in the other direction to deduce the recursive Bellman equation for the action value function

$$Q^\pi(s,a) = R(s,a) + \gamma\mathbb{E}_{s'\sim P(\cdot|s,a)}\left[\mathbb{E}_{a'\sim\pi(\cdot|s')}\left[Q^\pi(s',a')\right]\right]. \tag{2.6}$$

We are seeking an **optimal policy** π^* which is defined as a policy that achieves the highest possible expected return from every state so that $V^{\pi^*}(s) \geq V^\pi(s)$ for all states $s \in \mathcal{S}$ and all policies π. This gives us the corresponding optimal state value function $V^*(s) := \sup_\pi V^\pi(s)$ and optimal action value function $Q^*(s,a) := \sup_\pi Q^\pi(s,a)$. These optimal value functions describe the highest return that can be achieved from a given state or state-action pair if the agent acts according to an optimal policy thereafter. Crucially, knowing the optimal state value function is not sufficient to deduce an optimal policy unless we have access to the transition model to compute expectations. However if $Q^*(s,a)$ is known, we can define an optimal policy

$$\pi^*(s) \in \arg\max_{a\in\mathcal{A}} Q^*(s,a). \tag{2.7}$$

This equation is fundamental for a large number of control problems in reinforcement learning, where we seek an optimal policy. In particular, the tabular methods discussed in the following section and value-based deep RL covered in Chapter 3 rely on this expression of the optimal policy.

Now that we have built an understanding of policies, value functions and

what makes a given policy or value function optimal, we will now consider how these quantities may be estimated in practice. We begin with tabular learning methods, in which the state and action spaces are assumed to be finite, meaning we can explicitly represent value functions as tables. Although these methods are limited to relatively simple environments, they provide the conceptual foundation for many modern reinforcement learning algorithms that we will explore in the following chapter.

2.2 Tabular Methods

As mentioned earlier, dynamic programming (DP) methods are optimisation methods that can be used when the agent has full access to the exact transition and reward functions of the environment. Though they make strict assumptions and become computationally inefficient in large environments, they provide exact solutions to problems of this kind by using direct iteration of the Bellman equations. DP is exact in the sense that it guarantees asymptotic convergence to the optimal solution with no estimation uncertainty. This differs from the convergence of unbiased estimators discussed in Chapter 3 since the only error between the current estimate and the optimal value $||V_k - V^*||$ in DP comes from the finite number of iterations. It can be shown that the Bellman operator, which defines the update rule in DP, is a **contraction mapping** (Denardo, 1967; Kumar, 2020). A γ-contraction for $\gamma \in [0, 1)$ is defined as an operator f on a $||\cdot||$ normed vector space satisfying

$$||f(x) - f(y)|| \leq \gamma||x - y||$$

for all $x, y \in \chi$. This means we can bound

$$||V_{k+1} - V^*|| \leq \gamma||V_k - V^*||.$$

The Banach fixed-point theorem then tells us that repeated application of such a contraction mapping converges to a unique fixed point in χ, demonstrating the algorithm's asymptotic convergence.

If we do not have the model knowledge required for DP, but our state and action spaces are sufficiently small and discrete [3], we can employ the set of methods described below.

2.2.1 Monte Carlo and Temporal Difference Learning

Without explicit knowledge of the environment, the agent must learn from experience through environment interaction. This leads to two of the most fundamental classes of model-free methods, which both aim to learn the optimal action value function for each state-action pair in order to deduce an optimal policy. **Monte**

[3] If either of the sets is continuous, we can form a discretised grid to obtain an approximation of the underlying continuous set. Although due to the curse of dimensionality, this approximation is often too coarse to generate high-performing policies.

Carlo (MC)[4] methods update value function estimates using returns computed from complete sampled episodes. While **Temporal Difference (TD)** methods use a technique called bootstrapping to update the value function estimate using the immediate reward and the current value estimate of a subsequent state.

An agent learning with MC methods collects a full episode trajectory with the current policy, then for each visited state-action pair $s_t,\ a_t$, computes the return $G_t = \sum_{i=0}^{T-t-1} \gamma^i r_{t+i}$ from time t. This return is used to update the current estimate $Q(s_t, a_t)$ of $Q^\pi(s_t, a_t)$ with the update rule

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(G_t - Q(s_t, a_t)),$$

for a learning rate $\alpha > 0$. This moves the current estimate towards the observed return which acts as an unbiased estimator of the action value function under the current policy. However, since this approach requires observing a full trajectory, the value updates typically have very high variance due to the high number of stochastic processes occurring in the episode. Additionally, learning can be slow with a high computational cost since the episode must terminate before the return can be computed.

On the other hand, TD methods do not need to collect full episode returns, and instead update action value estimates by bootstrapping from the current value estimate of the future state. The simplest bootstrapping approach is called $TD(0)$, in which the agent observes a single environment step acting with the current policy and uses the target $r_t + \gamma Q(s_{t+1}, a_{t+1})$ as an approximation for G_t. This target gives us the SARSA update, an **on-policy** control method in which the value function is updated using information derived from the current policy. Q-learning is an alternative **off-policy** approach where $Q(s_{t+1}, a_{t+1})$ in the target is replaced with $\max_a Q(s_{t+1}, a)$. The $TD(0)$ SARSA update rule is therefore

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)].$$

Temporal difference methods tend to be much faster than MC methods as they update the value estimates at each step rather than being restricted to only updating once per episode. This also results in less required computation per update.

In non-stationary problems, it is clear that maintaining some form of exploration throughout training and deployment is essential, as changes in the environment dynamics will likely alter the optimal policy. However, even in stationary environments, the lack of knowledge that we have about the optimality of our current policy necessitates keeping some exploration during training. Therefore, instead of using equation (2.7) to define a policy from our computed estimate, one common method of action selection is the **ϵ-greedy** approach. This means that at each timestep, given an exploration parameter $\epsilon \in [0, 1]$, a random action $a \sim \text{Unif}(\mathcal{A})$ is selected with probability ϵ to ensure exploration is maintained,

[4] In statistics, Monte Carlo methods are used to approximate expectations with empirical averages. We collect samples $x_1, \ldots, x_N$ from a random variable X, then approximate $\mathbb{E}[X] \approx \frac{1}{N}\sum_{i=1}^{N} X_i$. Here, returns computed from sampled trajectories provide Monte Carlo estimates of expected returns.

while the 'greedy' action $a^* \in \arg\max_a(Q(s,a))$ is chosen with probability $1-\epsilon$. In the methods discussed so far, the Q value estimates produced, combined with the method of action selection, implicitly define the policy. In deployment or evaluation, we often use greedy policies with $\epsilon = 0$, which are deterministic versions of the stochastic policy learned when training.

These tabular methods are effective in simple environments where value functions can be represented explicitly for each state-action pair. However, they do not scale well to high-dimensional problems. Therefore, we will focus this dissertation on deep reinforcement learning.

Chapter 3

Deep Reinforcement Learning

For continuous or high-dimensional state/action spaces, we cannot practically use the tabular methods discussed earlier, which maintain an exact representation of the function being learned for each input value. Instead, we must rely on **function approximators** such as neural networks. Deep Reinforcement Learning is the study of RL algorithms that involve learning an optimal parameterisation of some function approximator. Broadly speaking, there are two categories that deep RL algorithms fall into: **policy gradient (PG) methods**, which are the focus of this dissertation, and **value-based methods**, which can be thought of as extensions of Q-learning discussed in the previous chapter. We will begin this chapter by justifying the focus of this report on stochastic policy gradient algorithms, which will then be combined with a Bayesian approach to RL, leading to the SVPG algorithm.

3.1 Value-based vs Policy Gradient Methods

Value-based methods learn a policy implicitly by learning an approximate action value function $Q_\phi(s, a)$, typically as a neural network parameterised by ϕ, which is then used to select actions and form an implicit policy. We will use a subscript Q_ϕ to represent such parameterisations of value functions expressed with a function approximator, a superscript Q^π to denote the true value function under policy π, and a superscript asterisk Q^* to represent the optimal value function. Similar to tabular TD methods, the policy often uses an ϵ-greedy approach to select actions from the learned value function. However, soft Q-learning (Schulman, Chen and Abbeel, 2017) and other entropy regularised methods can be used to promote action diversity and exploration.

One of the most notable examples of a value-based algorithm is DQN, which was first described in Mnih (2013) where it was shown to outperform other algorithms of the time with high-dimensional state spaces such as image pixels in Atari games. The algorithm was further developed and tested in Mnih et al. (2015). DQN uses a neural network parameterised by θ to approximate the optimal action-value function $Q_\theta(s, a) \approx Q^*(s, a)$. The network is trained by minimising the TD error between the predicted Q-values and the Bellman target $y = r + \gamma \max_{a'} Q_{\theta^-}(s', a')$ where θ^- represents the parameters of a separate target

network that is updated less frequently by copying $\theta^- \leftarrow \theta$. Data is collected by sampling trajectories from a separate behaviour policy, storing the trajectories in a replay buffer $U(D)$, and then drawing random minibatches $(s, a, r, s') \sim U(D)$ from the replay buffer. The loss function of the network at iteration i is then

$$L_i(\theta_i) = \mathbb{E}_{(s,a,r,s')\sim U(D)} \left[\left(r + \gamma \max_{a'} Q_{\theta_i^-}(s', a') - Q_{\theta_i}(s, a) \right)^2 \right].$$

This means that the network learns by repeatedly updating the Q network so that its predictions move closer to the moving target predictions. We will see similar ideas of updating a value function using TD targets when discussing actor-critic methods in Section 3.4.

The key drawback to value-based methods is that they are very difficult to generalise to continuous action spaces since, in their current form, that would require solving a non-trivial continuous optimisation problem at each step. This optimisation is required to select the greedy action $\arg\max_a Q_\theta(s, a)$ if using ϵ-greedy actions as well as in the Bellman target $y = r + \gamma \max_{a'} Q_\theta(s', a')$. In contrast, for discrete action spaces it is possible to simply enumerate each action and select the one with the largest estimated value. One way to get around this is to discretise the action space into a grid, but this makes the learning an approximation subject to the resolution chosen. Also, the curse of dimensionality tells us that the dimension of the action space can quickly explode as we increase the resolution, leading to extremely high computational cost if we wish to compute a more accurate approximation.

Policy gradient (PG) methods do not suffer from this problem and instead use a function approximator to explicitly define the policy function π_θ with parameters θ. We will see that some PG methods still learn an approximate value function in addition to learning the policy. However, the explicit policy representation is what separates them from value-based methods.

3.2 Stochastic vs Deterministic Policies

PG methods can be used to learn a stochastic policy $\pi_\theta(a|s)$ or a deterministic policy $\mu_\theta(s)$ as in DDPG (Lillicrap et al., 2015). We will only discuss the case of stochastic policies for several reasons explored below. However, it is worth noting that deterministic policies typically have lower gradient variance since they remove the uncertainty caused by sampling over actions. Silver et al. (2014) showed that the deterministic policy gradient depends on an expectation over the state distribution whereas we see that the stochastic policy gradient in equation (3.2) uses an expectation over trajectories of state action pairs. This difference also favours deterministic policies in high-dimensional action spaces where sampling over the actions becomes computationally expensive.

There are, however, several downsides to removing the stochasticity of the policy. The first of which is the clear difficulty of maintaining exploration. While stochastic policies can explore naturally with learned state-dependent uncertainty, deterministic policies require incorporating some external noise, which doesn't

represent the policy's true uncertainty in the current state. Deterministic policy gradient algorithms also introduce a level of bias in the gradient since practical application of the deterministic policy gradient theorem requires approximating $Q^\mu(s,a)$ with a separate network $Q_\phi(s,a)$ called a critic. This critic is an approximation, meaning $Q_\phi \neq Q^\mu$ hence the policy gradient is biased. There is additional bias in the off-policy DDPG algorithm, which approximates the expectation over $\rho_\mu(s)$ by sampling state distributions from a behaviour policy β over $\rho_\beta(s)$. Since this behaviour policy differs from the μ, the expectation will be biased. Finally, while in single-agent MDPs, there always exists an optimal deterministic policy (Puterman, 1990), there is value in learning the optimal stochastic policy since they are often more interpretable and provide us with information about the policy's uncertainty in certain states. This is particularly important for Markov games, which are multi-agent environments (Littman, 1994) where optimal deterministic policies may not exist. Learning a flexible strategy with optimal action probabilities in each state is vital, as a deterministic policy is likely exploitable.

The practical advantages of deterministic policies are not inherent to the determinism itself but arise from the reduced variance and stability of the estimates that these methods produce. The primary reason we will favour stochastic policies is that this report is most concerned with uncertainty quantification of RL problems. Deterministic policies discard valuable information about the uncertainty of the policy in certain states, which will be explored further in Chapter 4. Since the aforementioned advantages of deterministic policies can be found in certain variations of stochastic algorithms, which will be covered in the remainder of this chapter, we will assume a stochastic policy from now on.

3.2.1 Policy Representation

The policy $\pi_\theta(a|s)$ is typically represented by a neural network parameterised by a vector θ of weights and biases. For discrete action spaces $\mathcal{A} = \{1, \dots, n\}$, the network's final layer typically uses a softmax activation function to map the output logits $z(s) \in \mathbb{R}^n$ of the network to

$$\pi_\theta(A_t = i|s) = Pr(A_t = i|S_t = s, \theta) = \frac{e^{z_i(s)}}{\sum_j e^{z_j(s)}},$$

representing the probability of each action. The activation function ensures non-negativity and unit sum of probabilities and will be particularly relevant in Chapter 5 when we perform prior predictive checks over weights of the network and observe the resulting behaviour.

In contrast to this, for environments with a continuous action space $\mathcal{A} \subseteq \mathbb{R}^n$, we typically use a Gaussian policy where the network outputs a learned mean μ_θ and standard deviation σ_θ. The policy is then an n-dimensional normal distribution with diagonal covariance

$$a \sim \pi_\theta(\cdot|s) = \mathcal{N}_n(\mu_\theta(s), diag(\sigma^2_\theta(s))).$$

For bounded action spaces, we must alter this distribution so that all of the

probability mass is contained within the bounds. The simple solution is to sample from the Gaussian, and then clip the action to the allowed interval. However, this approach does not represent the true distribution induced by the policy. The alternative approach is to sample a vector $u \sim \mathcal{N}_n(\boldsymbol{\mu}_\theta(s), diag(\sigma^2_\theta(s)))$, and transform it using $a = \tanh(u)$ to ensure all resulting actions a are in the interval $(-1, 1)^n$. When computing $\pi_\theta(a|s)$, we must then change the variables back to get the correct Gaussian density using $u = \tanh^{-1}(a)$.

To reiterate the goal described in Chapter 2 in the context of policy gradient methods, the standard RL objective for PG methods is to find the θ that parametrises the policy $\pi_\theta(a|s)$ corresponding to the maximum expected returns

$$J(\theta) = \mathbb{E}_{\tau \sim \rho_{\pi_\theta}}[\sum_{t=0}^{T-1} \gamma^t r_t].$$

Here, $\tau \sim \rho_{\pi_\theta}$ represents a trajectory of states, actions and rewards taken by the agent using policy π_θ and we are taking the expectation over the distribution of possible trajectories induced by π_θ. As with most neural network based algorithms, we use a form of stochastic gradient descent (Bottou, 2010) in which we adjust θ to move in the direction of the **policy gradient** $\nabla_\theta J(\theta)$. We often define a loss function since maximising the expected return is equivalent to minimising $L(\theta) = -J(\theta)$, and allows us to utilise descent algorithms. We use the update rule

$$\theta' \leftarrow \theta - \alpha \nabla_\theta L(\theta) \tag{3.1}$$

for some learning rate $\alpha \in \mathbb{R}_{>0}$.

This is the foundation of a set of policy gradient algorithms where estimates of the policy gradient can be used to update the policy parameters without requiring an explicit model of the transition function. Of course, we do not know the exact value of the expected return $J(\theta)$ and nor can we compute its exact gradient. The remainder of this chapter will look at ways that we can estimate the policy gradient, including analysing the variance of the estimator and ways in which it can be reduced. We will see that the choice of estimator largely comes down to a trade-off between computational cost, bias and variance. Our choice of PG estimator is vital for Chapter 5 where we discuss our implementation of SVPG, a variational inference method which relies directly on selecting a suitable policy gradient estimator.

3.3 Policy Gradient Theorem and REINFORCE

The **policy gradient theorem**, originally derived in Sutton et al. (1999), gives us (Lehmann, 2024) (Peters, 2010)

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \rho_{\pi_\theta}}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) Q^{\pi_\theta}(s_t, a_t)]. \tag{3.2}$$

The first term is the gradient of the log likelihood of the chosen action with respect to the policy parameters. In practice, we can compute this exactly via backpropagation of the policy network using automatic differentiation. The action-value function, however, cannot be computed directly in most cases. It does have a closed form solution, as we saw in Chapter 2, but that solution is intractable for all but the simplest environments. The way in which we estimate the action value term dictates the nature of the policy gradient algorithm.

The most basic PG estimator has been used since before the derivation of the policy gradient theorem and defines the **REINFORCE** (Williams, 1992) algorithm. Since $Q^{\pi_\theta}(s_t, a_t) := \mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[G_t|S_t = s_t, A_t = a_t]$, the expected value of the random variable G_t computed from a sample trajectory using π_θ equals $Q^{\pi_\theta}(s_t, a_t)$ making G_t an unbiased estimator. In the equations below, we assume that all expectations are over trajectories $\tau \sim \rho_{\pi_\theta}$. Starting from the policy gradient theorem (3.2), we find that:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \mathbb{E}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) Q^{\pi_\theta}(s_t, a_t)] \\
&= \mathbb{E}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \mathbb{E}[G_t|s_t, a_t]] \\
&= \mathbb{E}[\sum_{t=0}^{T-1} \mathbb{E}[\nabla_\theta \log \pi_\theta(a_t|s_t) G_t|s_t, a_t]] \\
&= \mathbb{E}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t],
\end{aligned}$$

where we use the tower property of the expectation in the final line. This gives us the tractable policy gradient estimator

$$\widehat{g}^{\text{REINFORCE}} = \widehat{\nabla_\theta J(\theta)} = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t. \tag{3.3}$$

We can compute samples of $G_t = \sum_{i=0}^{T-t-1} \gamma^i r_{t+i}$ by collecting full trajectories under policy π_θ. The algorithm assumes episodic environments to allow computing a full return of an episode. For infinite horizons, we can make use of the discount factor which reduces the magnitude of more distant rewards, allowing us to ignore all returns after the n-th timestep. This is referred to as n step truncation and gives us the return $G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k}$. Truncation introduces bias since $G_t^{(n)} \neq G_t$ for any n less than the length of horizon length, and the variance of the estimator remains high. For this reason, TD methods described in Section 3.4 are more commonly used for infinite horizon problems.

Using the estimator (3.3) in the descent rule (3.1) gives us the REINFORCE update rule

$$\theta' \leftarrow \theta + \alpha \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) G_t.$$

While this estimator is unbiased by construction, it has incredibly high variance, as we will examine later, because computing a full return depends on many stochastic variables. The random transitions, starting states and reward function, as well as the stochastic nature of the policy, all contribute to the variance. With long trajectories and a $\gamma \approx 1$, this makes learning incredibly unstable.

3.3.1 Variance Reduction

There are several ways to reduce the variance of the Monte Carlo estimator above. We will provide a theoretical overview and computational analysis of these methods before comparing them to actor-critic methods, which are primarily used in modern RL. Our computational analysis of their effects on variance and learning stability will help us understand estimator noise, which is relevant in the following chapter, where we discuss forms of uncertainty and how Bayesian inference methods are used to analyse them.

Batched Learning

The first of these methods involves taking the trajectories from a **batch** of B episodes before performing an update of the policy parameters. If episode i has length T_i, we know the gradient estimator for that episode is

$$\widehat{g}^{(i)} = \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta(a_t^{(i)}|s_t^{(i)}) G_t^{(i)}.$$

After collecting B trajectories and computing their gradients $\{\widehat{g}^{(1)}, \widehat{g}^{(2)}, \ldots, \widehat{g}^{(B)}\}$, we can then choose to average over the number of episodes or average over the number of timesteps with the estimators

$$\widehat{g} = \frac{1}{B}\sum_{i=1}^{B} \widehat{g}^{(i)}, \quad \text{or} \quad \widehat{g} = \frac{1}{\sum_{i=1}^{B} T_i}\sum_{i=1}^{B} \widehat{g}^{(i)},$$

respectively. The former treats every episode equally, while the latter gives greater weight to longer episodes. Averaging over timesteps can be preferable when episode length varies, since the estimate is more closely related to the amount of data collected. However, averaging over the number of episodes is often used to ensure the estimator remains unbiased and to give a clearer interpretation of each trajectory contributing a single unbiased sample. In our experiments and further discussion, we will use the convention of averaging over the number of episodes.

The variance of each gradient estimator when collecting and averaging over multiple trajectories reduces by a factor of $\frac{1}{B}$ as we can see by

$$Var(\frac{1}{B}\sum_i g_i) = \frac{1}{B^2}\sum_i Var(g_i) = \frac{1}{B} Var(g).$$

We are assuming i.i.d gradient estimates, which is true since they are each computed from trajectories induced by the same policy. Choosing to only update the

policy after collecting B episodes will, of course, come with B times the computational expense per update. However, it is very likely that fewer total episode samples will be required to converge to a good policy due to the much lower variance, reducing the frequency of gradients pointing in a direction that worsens the policy. In fact, a batch size that is too small may prevent the policy from learning more complex problems entirely as demonstrated in Section 3.3.2.

Baseline Functions

Another key method of variance reduction involves subtracting a state-dependent **baseline function** $b(s)$ from the return to give the estimator

$$\widehat{g}(b) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t)). \tag{3.4}$$

We will first show that this estimator is unbiased, assuming the baseline has no dependence on a_t. Expanding the expected value of this estimator gives

$$\mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t))] = \nabla_\theta J(\theta) - \mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) b(s_t)].$$

We now take the sum over t to the outside of the expectation by the linearity of expectation and split the expectation over trajectories into an expectation over the marginal distributions over states and actions. Since the baseline is independent of a_t, the expectation can be rewritten

$$\begin{aligned}\mathbb{E}_{\tau\sim\rho_{\pi_\theta}}[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) b(s_t)] &= \sum_{t=0}^{T-1} \mathbb{E}_{s_t\sim\rho_{\pi_\theta},\ a_t\sim\pi_\theta(\cdot|s_t)}[\nabla_\theta \log \pi_\theta(a_t|s_t) b(s_t)] \\ &= \sum_{t=0}^{T-1} \mathbb{E}_{s_t\sim\rho_{\pi_\theta}}[b(s_t)\mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}[\nabla_\theta \log \pi_\theta(a_t|s_t)]],\end{aligned}$$

where, for discrete action spaces,

$$\begin{aligned}\mathbb{E}_{a_t\sim\pi_\theta(a_t|s_t)}[\nabla_\theta \log \pi_\theta(a_t|s_t)] &= \sum_{a_t} \pi_\theta(a_t|s_t) \nabla_\theta \log \pi_\theta(a_t|s_t) \\ &= \nabla_\theta \left(\sum_{a_t} \pi_\theta(a_t|s_t) \right).\end{aligned}$$

Finally, since $\pi_\theta(a_t|s_t)$ represents a probability distribution with unit sum over actions, the sum vanishes, hence the overall expectation vanishes, and the estimator is unbiased with expectation $\nabla_\theta J(\theta)$. The proof is analogous for continuous actions using integrals rather than sums.

The following sources (Mei et al., 2022; Chung et al., 2021; Lawrence et al., 2012) have studied the precise effect that the baseline has on the estimator variance. In particular, Greensmith et al. (2004) proves in Theorem 13, that the

optimal baseline function to minimise estimator variance is

$$b^*(s) = \frac{\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[||\nabla_\theta \log \pi_\theta(a|s)||^2 Q^\pi(s,a)]}{\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}[||\nabla_\theta \log \pi_\theta(a|s)||^2]}.$$

Since the exact value of $b^*(s)$ requires knowledge of the action value function and of expectations under the current policy, it is generally intractable in practice. Using equation (2.4), we see that $b(s) = V^\pi(s)$ resembles an unweighted representation of $b^*(s)$ and is therefore often used in implementations. Intuitively, using the value function as a baseline centres the returns, reducing the variability of the gradient estimator without changing its expectation.

Subtracting the value function from the returns leads us to the **advantage function**

$$A^\pi(s,a) := Q^\pi(s,a) - V^\pi(s)$$

which represents how much better taking action a while at state s is, compared to the value of the state under the current policy. With a baseline $b(s) = V^\pi(s)$, we have $Q^\pi(s,a) - b(s) = A^\pi(s,a)$ giving us the policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t) A^\pi(s_t,a_t)],$$

which we have shown to be unbiased. This means that actions we believe to perform better than our current policy have a positive weight on the gradient, while actions with lower than expected returns are assigned a negative weight. The idea of estimating the advantage function in order to compute the policy gradient will be explored further when we discuss GAE below, an algorithm used in our final implementation for performance when conserving the unbiased property of the estimator is no longer a concern.

3.3.2 Computational Analysis

We will now provide a short analysis of the variability of the bias-free Monte Carlo PG estimators discussed above, and the effect they have on the stability of learning. The purpose of this analysis is to show that subtracting a baseline function from the returns and computing a batch of trajectories meaningfully reduces the variance of the gradient estimate. The learning curves demonstrate that this reduction in variance is often required to solve more complex learning problems.

For these experiments, we will use the Lunar Lander (Farama Foundation, 2024*b*) environment, which has discrete actions and dense rewards guiding the agent to a fairly complex behaviour. We will first produce a random policy and train a neural network representing a state-dependent baseline function using this policy to approximate $V^\pi(s)$. The training is performed by collecting a large number of samples from the environment, and then using regression to fit the observed states to their corresponding returns. Since $\mathbb{E}_{\pi_\theta}[G_t|S_t = s_t] = V^{\pi_\theta}(s_t)$, our baseline function is an unbiased estimator of the state value function. After training the network, we compute the average observed return and use this to

define the baseline b_{fixed}. We then compute 500 policy gradient estimates using the same policy with each of the following estimators.

1. Default REINFORCE with no baseline
2. Subtracting a constant fixed baseline b_{fixed}
3. Subtracting a learned, state-dependent baseline $b(s)$.

Each of the three baseline estimators is identically seeded for a given sample, meaning their trajectories are identical and comparable. Additionally, each will be tested using a batch size of $B = 1$ and $B = 5$ for a total of 6 gradient estimator distributions. Our baseline network was trained on $n = 2000$ sampled trajectories until it converged to $R^2 \approx 0.717$ after around 35 epochs. We found that increasing the number of training samples did not improve the R^2 value beyond this point. Additionally, the train and validation MSE were very similar, suggesting no overfitting and the error in the regression coming from the noisy Monte Carlo estimates.

We decided to compare the variance of the gradient estimators by using the metric $Var(u^T g)$ for a fixed random projection vector u rather than $Var(||g||)$. This is because the latter is nonlinear in the components of g, meaning its variance can increase even if the variance of the individual components of g decreases. Using a projection vector gives a scalar metric which is linear in the components of g and whose variance,

$$Var(u^T g) = u^T Cov(g) u,$$

depends directly on the covariance of g. This provides a more reliable reflection of the changes in variance and is easy to compute.

The results of our experiment are shown in Figure 3.1 and Table 3.1, where we can clearly see that both introducing a baseline function and increasing the batch size have a meaningful effect on variance reduction. As expected, our fixed constant baseline performed worse than the learned, state-dependent baseline. This suggests that learning a state-dependent baseline is more effective at reducing variance than using a state-independent baseline from the same data. This is a key observation and motivates the use of a critic in the following section.

Estimator	$B = 1$	$B = 5$
None	6.105	1.179
Constant (fixed)	1.548	0.3386
Learned baseline	0.7442	0.1623

Table 3.1: Variance $\text{Var}(u^T g)$ for different baseline choices and batch sizes B for a random, fixed projection vector u.

It is worth noting that if we were training an agent and updating the policy, rather than just collecting PG estimates from a fixed policy, as the policy updated, so would the value function estimates. This means the originally

trained baseline function would less accurately approximate $V^{\pi_\theta}(s)$ and must be re-estimated in order not to contribute to additional variance. This idea links directly to **actor-critic methods** discussed in the following section. Using noisy Monte Carlo returns rather than the lower variance bootstrapped estimates in actor-critic methods means the critic must be trained a large number of times at each policy update to maintain a reasonable approximation of the value function without adding variance. This results in an unfeasibly high computational cost, which is why it is not performed in practice.

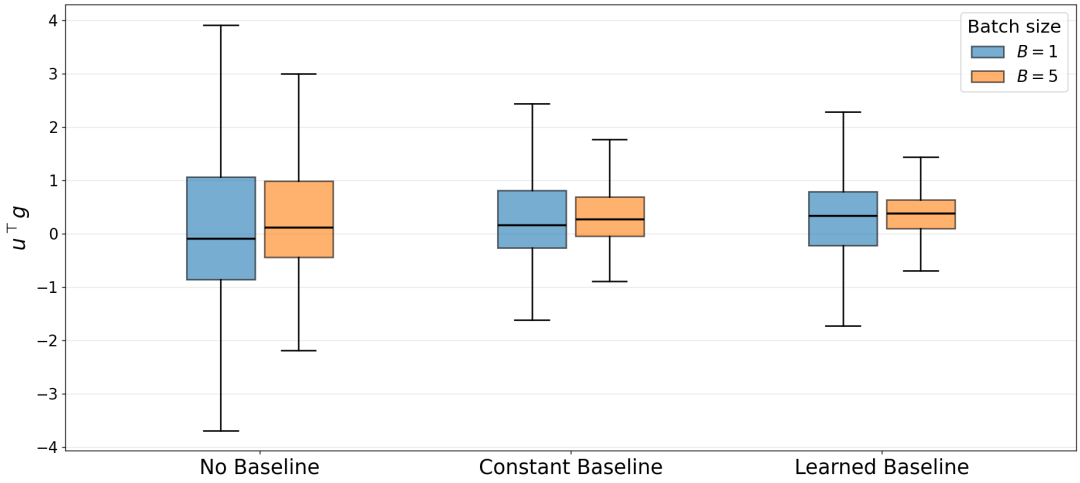

Figure 3.1: Distribution of $\mathrm{Var}(u^T g)$ for different baseline choices and batch sizes B for a random, fixed projection vector u.

We will now demonstrate the effect that this variance reduction has on learning by training agents on the Cartpole (Farama Foundation, 2024*a*) environment, as well as the same Lunar Lander environment. We will compare the effect of changing the batch size $B = 1$ and $B = 5$, as well as the effect of subtracting a constant baseline function $b_{\text{const}} = \frac{1}{T_{tot}} \sum_{i=1}^{B} \sum_{t=0}^{T_i - 1} G_t^{(i)}$, where T_i is the length of episode i and $G_t^{(i)}$ represents the observed returns from the current batch of episodes. This is similar to b_{fixed} from the previous experiment, except we now updated the constant baseline on the returns collected from the current episode. This baseline was chosen due to the advantages that updating the baseline function brings, and the challenges of using an updating learned baseline that we mentioned. We run a total of 500 iterations for each method and plot the rolling average return over the last 30 iterations. For consistency, this is the form we maintain with all learning curves in this dissertation. The results are then averaged over 5 seeds with the standard deviation between seeds plotted as a band around the average. The results for the Cartpole and Lunar Lander experiments are shown in 3.2 and 3.3, respectively.

As expected, the estimators that we observed to have the lowest variance are capable of achieving higher returns in the more complex environment and faster, more stable learning in both environments. The purpose of displaying the comparison on Cartpole is to show that while a batch size of 5 converges in significantly fewer policy updates, it requires more total environment samples

and is therefore more computationally expensive in this case. The agent with a batch size of 1 is capable of achieving the maximum return in fewer environment samples. This tells us that for simple environments, when speed is the primary concern, using a smaller batch size can be superior. However, this is not always the case, as evidenced by the Lunar Lander environment, where the $B = 1$ agent was unable to reach the goal return of 200 and, in fact, struggled to break past 0.

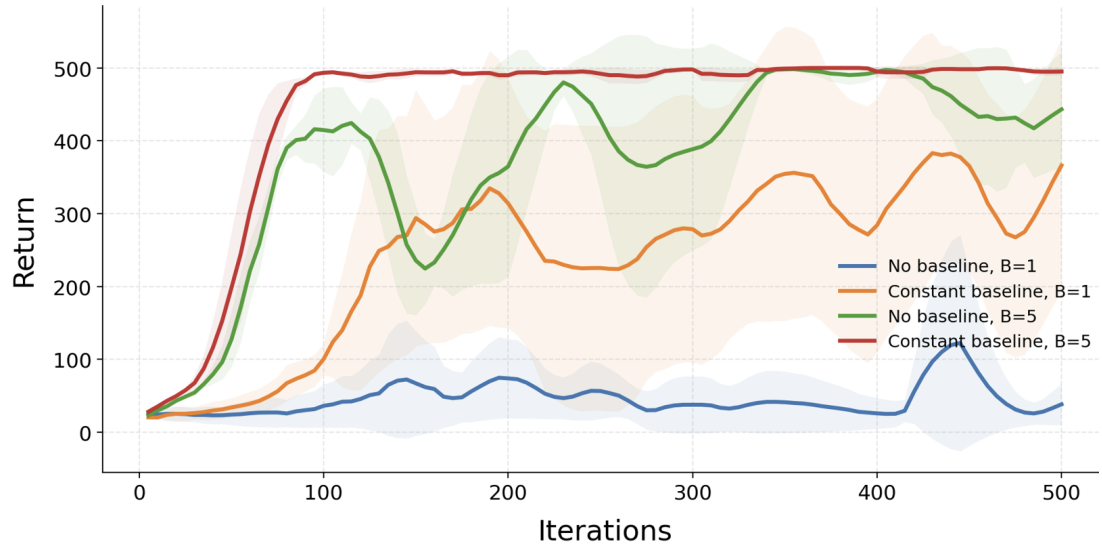

Figure 3.2: Learning curves for different REINFORCE variants using the Cartpole environment

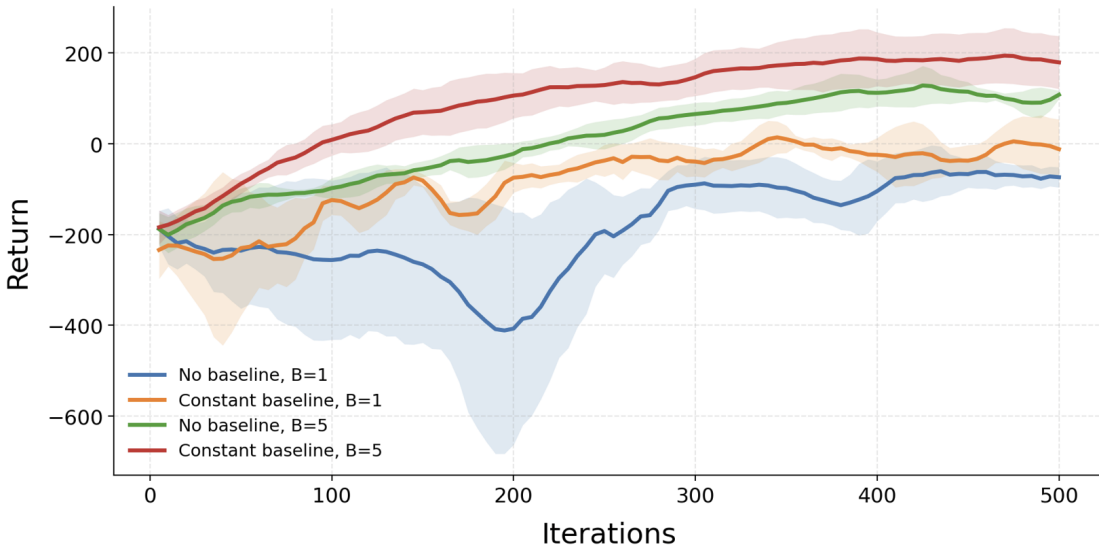

Figure 3.3: Learning curves for different REINFORCE variants using the Lunar Lander environment

3.3.3 Summary

To conclude this section, we have found that using pure Monte Carlo estimates to estimate the policy gradient produces an estimator with exceptionally high variance. We have explored and tested several methods to reduce this variance while maintaining the appealing unbiased nature of the estimator. However, we can see that it can be computationally expensive to do so, and the variance often remains high, hence learning remains unstable. Additionally, for more complex tasks, such as those with a sparse reward signal, the noise in the estimator is overwhelmingly high, and learning is nearly impossible. Because of this, we will move our attention back to temporal difference learning introduced in Chapter 2 in the form of actor-critic methods. We will see that these methods dramatically reduce the variance of the estimator at the cost of introducing bias.

3.4 Actor-Critic Methods

We will now introduce the concept of actor-critic methods (Grondman et al., 2012) for policy gradient estimation. This involves using an **actor** π_θ, which represents the policy that we use to sample from the environment as before, and a **critic network** parameterised by ϕ which maintains an approximation of a value function such as $V^\pi(s)$, $Q^\pi(s,a)$ or $A^\pi(s,a)$. We saw at the end of the previous section that subtracting a baseline function $b(s) \approx V^\pi(s)$ from the sampled returns is an effective method of reducing variance. However, as mentioned, when the policy changes during training, the true value function $V^{\pi_\theta}(s)$ also changes, and the baseline must be continuously retrained. Maintaining an approximation of the value function throughout training is precisely the role of a critic. Additionally, since the value function has no dependency on the action taken, our estimator will remain unbiased if we continue collecting full trajectories to compute G_t. However, using Monte Carlo estimates to train the critic would require us to wait until the end of each trajectory before updating the value function, which leads to extremely high variance updates. We observed this when training $b(s)$ in our previous experiment where our network struggled to achieve a high R^2 value. Collecting a large enough number of trajectories to update the critic without incredibly high variance would have an infeasibly high computational cost. For this reason, in practice, actor-critic methods typically use temporal difference (TD) updates instead of Monte Carlo returns to estimate value targets and to train the critic.

Actor-critic methods typically use the critic network to approximate the value function $V_\phi(s) \approx V^\pi(s)$, although algorithms such as Haarnoja et al. (2018) and Lillicrap et al. (2015) use a $Q_\phi(s,a)$ critic. Instead of computing the full Monte Carlo return G_t which requires collecting all returns until the end of the episode, we can estimate it at each timestep using $G_t \approx r_t + \gamma V_\phi(s_{t+1}) := G_t^{(1)}$ where $\gamma V_\phi(s_{t+1})$ represents our discounted estimate of the value of the next state. This technique is analogous to the tabular TD methods in Chapter 2. The bootstrapped estimate of the returns is where the bias in our PG estimator is introduced since our approximate value function $V_\phi(s)$ is not generally equal to the true

$V^\pi(s)$ hence the estimated returns $\mathbb{E}_\pi[r_t+\gamma V_\phi(s_{t+1})|S_t = s_t, A_t = a_t] \neq Q^\pi(s_t, a_t)$. However, the variance is significantly lower because we are no longer sampling a long, noisy trajectory but using information that we have learned in our critic.

This is an example of **one-step TD** and we can decrease the bias and increase the variance by collecting more environment steps before bootstrapping by computing the more general **n-step TD target**

$$G_t^{(n)} = r_t+\gamma r_{t+1}+\cdots+\gamma^{n-1}r_{t+n-1}+\gamma^n V_\phi(s_{t+n}) = \sum_{k=0}^{n-1}\gamma^k r_{t+k}+\gamma^n V_\phi(s_{t+n}). \quad (3.5)$$

We can see that as $n \to \infty$, or even just for $n \geq T-t$, where T is the length of the episode, our n-step TD target $G_t^{(n)} = G_t$ is exactly the Monte Carlo return. This shows the increase of variance and decrease of bias that comes with increasing the number of timesteps we collect before using our critic to approximate the remaining steps.

The difference between our value target $G_t^{(n)}$ and our prediction for the value of the current state defines the **n-step TD residual**

$$\delta_t^{(n)} = G_t^{(n)} - V_\phi(s_t) = \sum_{k=0}^{n-1}\gamma^k r_{t+k} + \gamma^n V_\phi(s_{t+n}) - V_\phi(s_t). \quad (3.6)$$

This residual represents a **biased** estimate of the advantage function. The key distinction between this and the estimator (3.4) with a learned baseline $b(s) \approx V^\pi(s)$ is that now we are computing a bootstrapped estimate of the action value function instead of an unbiased estimate, reducing variance and introducing bias. This additional bias removes the convergence guarantee that we had with REINFORCE, since with an unlimited number of samples, we may converge to a suboptimal solution despite the lower variance likely speeding up this convergence. Now using our advantage function estimate in (3.2) we find the policy gradient estimator

$$\widehat{g}^{TD(n)} = \sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t|s_t)\delta_t^{(n)}.$$

Now that we do not need to collect the trajectory from a full episode, we can update θ incrementally every timestep once we have collected the first n steps. This gives the update rule for the actor

$$\theta' \leftarrow \theta + \alpha\nabla_\theta \log \pi_\theta(a_t|s_t)\delta_t^{(n)} \quad (3.7)$$

Updating the policy during an episode is known as **online** learning and has many practical advantages, especially for very long or even infinite length environments.

So far, we have talked about how we use our critic to estimate the advantage function and update the actor's parameters. The critic network is trained through stochastic gradient descent using the same bootstrapped value target that we used

for the actor, with the loss function

$$L(\phi) = \frac{1}{2}(G_t^{(n)} - V_\phi(s_t))^2 = \frac{1}{2}(\delta_t^{(n)})^2.$$

The critic network aims to reduce this loss function by making the value estimates as close to the bootstrapped target $G_t^{(n)}$ as possible. This corresponds to reducing the size of the estimated advantage function. With our loss function, we compute the gradient, assuming $\nabla_\phi G_t^{(n)} = 0$ because our TD target is constant, giving us

$$\nabla_\phi L = \nabla_\phi \left(\frac{1}{2}(G_t^{(n)} - V_\phi(s_t))^2\right) = (G_t^{(n)} - V_\phi(s_t))\nabla_\phi(G_t^{(n)} - V_\phi(s_t)) = -\delta_t^{(n)}\nabla_\phi V_\phi(s_t)$$

using the chain rule and the definition of $\delta_t^{(n)}$. Unlike the policy gradient, this gradient is exactly computable using the network output, the returns from the current n timesteps to generate the target, and backpropagation to compute the critic gradient. Therefore, employing the same stochastic gradient descent method as before, the critic parameter update is

$$\phi' \leftarrow \phi + \beta\delta_t^{(n)}\nabla_\phi V_\phi(s_t), \tag{3.8}$$

for a separate learning rate $\beta > 0$.

In summary, actor critic methods sample a small number of steps from the current policy, then use the returns from these steps to compute TD residuals. These residuals are used at each timestep to simultaneously update the critic's parameters using the TD loss function in equation (3.8), and update the actor's parameters in equation (3.7) using the residual as a biased estimator for the action value function in the policy gradient theorem. This represents the most basic form of an actor critic method, and extensions of this class of algorithm make up a significant amount of modern RL research due to the fast convergence they have been shown to provide in complex environments. We will not spend time investigating these more complex methods as they veer off from our goal of applying PG estimation to variational inference. Instead, we will consider GAE, a method of controlling the bias-variance tradoff of the advantage estimator by exponentially weighing each of the TD(n) updates. This method is directly relevant to our experiments in Chapter 5 as we use it in our implementation of SVPG to compute a low variance estimator with a small amount of bias.

3.4.1 Generalised Advantage Estimation

Generalised Advantage Estimation (GAE) (Schulman, Moritz, Levine, Jordan and Abbeel, 2015; ApX Machine Learning, 2026) uses the one-step TD residual

$$\delta_t^{(1)} = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t),$$

which alone acts as a very low variance but high bias estimator for the advantage function. As discussed above, we can reduce the bias of this estimator by collecting more steps before bootstrapping. Instead of this, GAE combines the

estimates of many one-step TD residuals evaluated at different timesteps, giving the exponentially weighted sum

$$\widehat{A}_t^{GAE(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^{(1)},$$

where γ is the standard discount factor. The value of $\lambda \in [0, 1]$ controls the tradoff between bias and variance with smaller values representing the TD errors from later timesteps having a lower influence. This corresponds to a lower variance and greater bias, where at the limit, $\lambda = 0$,

$$\widehat{A}_t^{GAE(0)} = \delta_t^{(1)}.$$

We can see that this is exactly the TD(0) advantage estimate used in the previous section. Alternatively, when $\lambda = 1$, we see

$$\begin{aligned}
\widehat{A}_t^{GAE(1)} &= \sum_{l=0}^{\infty} \gamma^l \delta_{t+l}^{(1)} \\
&= \sum_{l=0}^{\infty} \gamma^l (r_{t+l} + \gamma V_\phi(s_{t+l+1}) - V_\phi(s_{t+l})) \\
&= \sum_{l=0}^{\infty} \gamma^l r_{t+l} + \sum_{l=1}^{\infty} \gamma^l V_\phi(s_{t+l}) - \sum_{l=0}^{\infty} \gamma^l V_\phi(S_{t+l}) \\
&= \sum_{l=0}^{\infty} \gamma^l r_{t+l} - V_\phi(s_t),
\end{aligned}$$

where the last two sums telescope. This results in the Monte Carlo returns with a value function critic exactly as we estimated the advantages in Section 3.3.2. This shows us that GAE smoothly interpolates between the high bias TD(0) updates and the high variance MC updates, allowing us to tune our choice of λ most suitable for the current problem.

3.5 Conclusion

To conclude this chapter on deep reinforcement learning, specifically policy gradient estimators and algorithms, we have motivated the desire to compute a low variance PG estimator and shown computationally the effect this has on the stability of learning. We examined several bias-free methods and determined that their variance and/or computational cost remain high. Finally, we looked at actor-critic methods, which have significantly reduced variance but involve a biased objective and GAE, which interpolates between the two and which we incorporate into our final implementation.

Since our goal is not to find the policy gradient algorithm that leads to the best possible returns, we will not spend any time discussing the most modern algorithms. Currently, the 'state of the art' policy gradient algorithms used by

companies such as OpenAI are Proximal Policy Optimisation (Schulman, Wolski, Dhariwal, Radford and Klimov, 2017) and Soft Actor Critic (Haarnoja et al., 2018). The former incorporates GAE and then defines its own policy update rule, making it incompatible with SVPG, and the latter uses an off-policy entropy-regulated approach, which is also incompatible with our goal.

We will now move our discussion towards Bayesian inference methods and the use of prior regularisation, which will later be combined with PG methods just discussed to form the algorithm of interest to this report.

Chapter 4

Bayesian Inference for Reinforcement Learning

The previous chapter dealt primarily with algorithms that compute a point estimate of the policy gradient and then use this estimate to iteratively update a single parameter vector θ to maximise the expected returns. While these methods have been shown to achieve high returns in numerous environments, there are many sources of uncertainty that can arise during the process. Computing single estimates makes it difficult to know how much uncertainty remains in our final prediction and how confident we can be in specific actions dictated by the policy that we learned.

This chapter takes a closer look at each source of uncertainty in standard policy gradient algorithms and motivates the need to develop methods for uncertainty quantification. We then introduce the Bayesian perspective, in which uncertainty is represented through distributions over quantities of interest rather than point estimates. This leads to the role of prior distributions in Bayesian theory, both as a method to encode genuine prior beliefs and for regularisation to improve stability. Finally, we discuss variational inference (VI) (Blei et al., 2017) as a method for approximate Bayesian inference in settings such as RL, where exact posteriors cannot be computed. In particular, we explore Stein Variational Gradient Descent as a particle-based VI tool used to derive Stein Variational Policy Gradient, the main algorithm of interest for this dissertation.

4.1 Uncertainty in Policy Gradient Methods

We have seen in detail that the PG estimator contains a significant amount of uncertainty. However, when training an agent, we only have access to the observed returns and resulting parameter updates, making it difficult to know which components of the algorithm contribute most to overall uncertainty. The sources of uncertainty that arise during training can be roughly categorised into uncertainty in the learning process and uncertainty in the acting process.

Firstly, the stochastic nature of the environment means that for fixed policy parameters, different trajectory samples can lead to vastly different gradient estimates. This is an issue that we explored directly in section 3.3.2. A second

source arises from the optimisation process itself, where the parameters produced by the algorithm $\hat{\theta}$ will generally not be the true optimal parameters θ^*. This uncertainty is caused by not only the variance in the gradient estimates but also the finite number of optimisation steps or bias in the objective or gradient estimates. Even if we were to compute the optimal θ^* model parameters, the policy π_{θ^*} will generally not represent the true optimal policy since the model itself is an approximation. This model uncertainty $\pi_{\theta^*} \neq \pi^*$ demonstrates the fact that uncertainty in deep RL is not only a consequence of finite data and stochastic optimisation methods, but also related to the finite structure of the function approximator used.

These sources of uncertainty arise during the learning process and represent **epistemic uncertainty** that can theoretically be removed with enough information and a complex enough model. In addition to this, uncertainty can arise during the acting process. Even under the optimal policy π^*, the optimal action may not be known with certainty due to the stochasticity in the transition function and the reward function. Even if the action taken is indeed the action with the highest expected return, it can be the case that the realised return is lower than the realised return of taking a suboptimal action. This uncertainty cannot be removed with additional information and is referred to as **aleatoric uncertainty**. It is for this reason that stochastic policies are particularly important when attempting to quantify aleatoric uncertainty, as they provide direct representations by returning probabilities for each action.

Classical RL algorithms typically produce point estimates of value functions or policy parameters. This removes our ability to quantify these different sources of uncertainty, and thus, we have no representation of how much uncertainty remains in our final estimate. Introducing a Bayesian perspective to RL addresses this issue by working with distributions over quantities rather than point estimates.

4.2 Bayesian Inference

Standard Bayesian inference involves updating the distribution containing our uncertainty in an unknown parameter θ as we accumulate additional data $\mathcal{D}$. We call this distribution the **posterior** $p(\theta|\mathcal{D})$ and it is formed using a prior $p_0(\theta)$ and a likelihood function $p(D|\theta)$. Bayes' Theorem then gives us

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p_0(\theta)}{p(\mathcal{D})} \propto p(\mathcal{D}|\theta)p_0(\theta). \tag{4.1}$$

The marginal density of observations $p(\mathcal{D})$, called the **evidence**, is computed using the joint density

$$p(\mathcal{D}) = \int p(\theta, \mathcal{D})\ d\theta = \int p(\mathcal{D}|\theta)p_0(\theta)\ d\theta.$$

In many situations, such as reinforcement learning, this evidence is practically intractable in closed form. If we would like to compute expectations with the posterior distribution, we require the proportionality constant of the evidence.

Because of this, methods such as MCMC and Variational Inference are used to approximate the true posterior, allowing us to compute expectations, as well as other statistical properties of the distribution. Even though the evidence, and therefore the normalised posterior, is intractable, the gradient of the log posterior is often possible to compute since the evidence does not depend on θ. We see

$$\nabla_\theta \log p(\theta|\mathcal{D}) = \nabla_\theta \log p(\mathcal{D}|\theta) + \nabla_\theta \log p_0(\theta),$$

which will be used in SVGD where we approximate $p(\theta|\mathcal{D})$ using variational inference.

Since our primary interest in this dissertation is to analyse the behaviour of different prior distributions, we must make explicit what a prior is and the various purposes that it can have in machine learning.

4.2.1 Prior Distribution

Generally speaking, a prior distribution represents beliefs about plausible values of a parameter before any data is observed. In some cases, the prior is chosen to encode genuine information we have about the likely distribution of the parameter, referred to as an **informative prior**. The alternative use for the prior distribution that we will look at is to act as a regulariser, encouraging the model to produce simple solutions and acting as a stabilising term. We will analyse both types of prior information in the following chapter and specific forms of regularisation with the priors that we test. We find that prior regularisation in SVPG does not act like regularisation in standard Bayesian inference and in other forms of machine learning. To highlight the difference, we will discuss a common example of prior regularisation in supervised learning.

Ridge regression (kjytay, 2018), is a regularisation method used for linear regression in supervised learning. We model data as $y_i = x_i^T\beta + \epsilon_i$ where $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$ represents the noise in our observations. This gives us the distribution $y_i|x_i, \beta \sim \mathcal{N}(x_i^T\beta, \sigma^2)$ and the likelihood function

$$p(\mathcal{D}|\beta) \propto \exp(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - x_i^T\beta)^2).$$

Ridge regression then places a zero mean Gaussian prior on the parameter β so

$$p_0(\beta) = \mathcal{N}(0, \tau^2 I).$$

Using Bayes' theorem (4.1), this results in the posterior

$$p(\beta|\mathcal{D}) \propto p(\mathcal{D}|\beta)p_0(\beta) \propto \exp(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - x_i^T\beta)^2)\exp(-\frac{1}{2\tau^2}||\beta||_2^2).$$

We would like to find the mode of this posterior distribution, known as the **MAP**

estimate, in order to determine the most likely value of β. This corresponds to

$$\hat{\beta} = \arg\max_{\beta} p(\beta|\mathcal{D}) = \arg\min_{\beta} \sum_{i=1}^{n} (y_i - x_i^T \beta)^2 + \frac{\sigma^2}{\tau^2} ||\beta||_2^2,$$

giving us the ridge regression objective.

We see that in this formulation, the Gaussian prior acts as a regulariser by discouraging large coefficient values with the $\lambda||\beta||_2^2$ term, which can improve stability in the estimates of β. The most important observation of the MAP estimate above, with regards to this dissertation, is that as we increase the number of datapoints n, the likelihood becomes more sharply peaked and $\sum_{i=1}^{n}(y_i - x_i^T\beta)^2$ increases. This results in the relative effect of the prior regulariser decreasing, allowing the likelihood term to dictate the most probable location of β. Therefore, we see that in standard Bayesian inference, weakly informative priors can have a stabilising effect early in training without dominating the posterior once sufficient data is observed. The distinction between this result and the effect observed when using a similar weakly informative prior in SVPG is the main result of this dissertation.

4.3 Variational Inference

Variational inference (Blei et al., 2017) is an approximation method that allows us to compute approximate statistical properties of an intractable posterior distribution. While we saw in the ridge regression example that the MAP estimate can be computed without needing posterior approximation, other properties, such as the posterior mean, require the full, normalised posterior. Instead of sampling from the true posterior with methods such as MCMC, we seek a distribution $q(\theta) \in \mathcal{Q}$ from a family of tractable candidate distributions $\mathcal{Q}$ which is close in KL distance [1] to the true posterior by optimising

$$q(\theta) = \arg\min_{q(\theta)\in\mathcal{Q}} D_{KL}(q(\theta)||p(\theta|\mathcal{D})).$$

If we did not restrict ourselves to the family $\mathcal{Q}$, the solution to this optimisation problem $q^*(\theta)$ is exactly the posterior distribution since the relative entropy is minimised when $q(\theta) = p(\theta|\mathcal{D})$.

Variational inference has been shown to be a more suitable method of Bayesian inference when datasets are very large or models are complex due to faster convergence of optimisation methods compared to sampling methods. However, while MCMC methods converge asymptotically to the exact posterior distribution under suitable assumptions (Blei et al., 2017), VI only converges to the best approximation from the chosen family $\mathcal{Q}$. This means that once optimised, we can only say that $q^*(\theta)$ is the best posterior approximation within $\mathcal{Q}$.

[1] Kullback-Leibler divergence or relative entropy $D_{KL}(P||Q) = \sum_{x\in\mathcal{X}} P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_{x\sim P}[\log P(x) - \log Q(x)]$ is a non-negative, asymmetric (so not a distance metric) measure of the distance between two probability distributions. For continuous distributions, the summation above is replaced with an integral.

4.3.1 Stein Variational Gradient Descent

Stein variational Gradient Descent (SVGD) Liu and Wang (2016) is a variational inference algorithm which uses a set of 'particles' as approximation samples. Instead of parametrising the posterior approximation $q(\theta)$, it uses a deterministic update rule to move particles $\theta_1, \ldots, \theta_M$ such that they more closely approximate a target distribution $p(\theta|\mathcal{D})$. While it is still a variational inference method in the sense that it optimises a distribution over parameters, it does not require selecting a family of distributions $\mathcal{Q}$. As we discussed, this choice of $\mathcal{Q}$ is what can make VI methods non-exact. The algorithm uses gradient descent to minimise the KL divergence between the empirical distribution of particles $q(\theta)$ and the true posterior $p(\theta|\mathcal{D})$ as with standard VI. Each particle θ_i is updated at each step by the rule

$$\theta_i \leftarrow \theta_i + \epsilon\phi(\theta_i) \tag{4.2}$$

for a step size ϵ. The direction $\phi(\theta_i)$ in which each particle is moved can be represented as the solution to the optimisation problem

$$\phi^* = \arg\max_{\phi\in\mathcal{F}} -\frac{d}{d\epsilon} D_{KL}(q_\epsilon||p). \tag{4.3}$$

Here, q_ϵ is the distribution of θ after moving each of the particles using the transformation (4.2). Therefore, the optimal direction to move the particles is the steepest direction where moving all θ along it brings our approximate distribution closer to the true posterior. The term $\mathcal{F}$ represents the class of candidate functions over which the optimisation is performed. The closed form solution of this optimisation problem requires restricting $\mathcal{F}$ to a unit ball in a **reproducing kernel Hilbert space** (RKHS) $\mathcal{H}$, which is a Hilbert space of functions associated with a kernel function $k : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$. This kernel function dictates how particles interact during optimisation and measures the similarity between two particles. A common kernel function is the radial basis function (RBF) kernel

$$k(\theta_i, \theta_j) = \exp\left(-\frac{||\theta_i - \theta_j||^2}{h}\right)$$

for a bandwidth parameter $h > 0$. We observe that the kernel has a maximum of 1 when $\theta_i = \theta_j$ and decreases as the particles spread further apart with a gradient

$$\nabla_{\theta_j} k(\theta_i, \theta_j) = 2\frac{(\theta_i - \theta_j)}{h} k(\theta_i, \theta_j).$$

Now, under the restriction $F \in \mathcal{H}$, the closed form solution to (4.3) is given by

$$\phi^*(\theta) = \mathbb{E}_{\theta'\sim q}[k(\theta', \theta)\nabla_{\theta'} \log p(\theta'|\mathcal{D}) + \nabla_{\theta'} k(\theta', \theta)].$$

Replacing the expectation with the empirical mean of the particles gives us the Stein Variational Gradient

$$\phi^*(\theta_i) = \frac{1}{M}\sum_{j=1}^{M}[k(\theta_j, \theta_i)\nabla_{\theta_j} \log p(\theta_j|\mathcal{D}) + \nabla_{\theta_j} k(\theta_j, \theta_i)]. \tag{4.4}$$

We can see that the first term of the gradient, $k(\theta_j, \theta_i)\nabla_{\theta_j} \log p(\theta_j|\mathcal{D})$, moves particles in the direction of regions of higher probability. This higher probability is determined by a weighted average of the gradients evaluated at each of the other particles, representing information sharing. The second term, $\nabla_{\theta_j} k(\theta_j, \theta_i)$, is the repulsion term and points in the direction that increases the distance between nearby particles. This ensures that particles spread out to approximate the full posterior rather than collapsing towards a single mode. Importantly, using a single particle, the kernel $k(\theta_i, \theta_i) = 1$ and $\nabla_{\theta_i} k(\theta_i, \theta_i) = 0$. Therefore

$$\phi^*(\theta_i) = \nabla_{\theta_i} \log p(\theta_i)$$

meaning the single particle will move to approximate the MAP.

4.3.2 Stein Variational Policy Gradient

Now that we have built up an understanding of policy gradients, Bayesian variational inference and SVGD, we can introduce the primary topic of this dissertation. In this chapter, we will give an overview of the SVPG algorithm and the motivation for using a particle-based VI method in RL for optimisation. The following chapter will analyse the alternative Bayesian interpretation of the algorithm and suggest reasons that the interpretation may not hold.

Stein Variational Policy Gradient (SVPG) uses the particle-based updating of SVGD within reinforcement learning by maintaining a set of policy parameters θ_i referred to as policy particles. In line with the topic of this chapter, rather than optimising a single parameter vector, SVPG aims to learn a distribution over these policy particles to approximate a given distribution.

The distribution that we seek to optimise arises from the optimisation problem

$$\max_q \{\mathbb{E}_{\theta \sim q}[J(\theta)] - \alpha D(q||q_0)\},$$

where $q_0(\theta)$ represents a prior reference distribution and the **temperature parameter** α controls the relative weight of the prior and objective function. The solution to this optimisation problem is

$$q^*(\theta) \propto \exp(\frac{1}{\alpha} J(\theta)) q_0(\theta), \tag{4.5}$$

where the paper loosely claims that $\exp(\frac{1}{\alpha} J(\theta)$ acts as the likelihood function and $q(\theta)$ as the posterior. The algorithm approximates this target distribution using a set of M policy particles $\{\theta_i\}_{i=1}^M$ representing samples of the target distribution $q(\theta)$, which are then optimised using the SVGD algorithm (4.4) with target distribution (4.5). We can compute $\log q(\theta) = \frac{1}{\alpha} J(\theta) + \log q_0(\theta) + \log Z$ where Z is the normalising constant independent of θ. This gives us the update rule $\theta_i \leftarrow \theta_i + \epsilon \Delta\theta_i$ for particle θ_i given M particles and a learning rate/step size ϵ as

$$\Delta\theta_i = \frac{1}{M}\sum_{j=1}^{M}\left[\nabla_{\theta_j}\left(\frac{1}{\alpha}J(\theta_j) + \log q_0(\theta_j)\right)k(\theta_j,\theta_i) + \nabla_{\theta_j}k(\theta_j,\theta_i)\right]. \tag{4.6}$$

This update rule contains three key components relevant to the RL process. Firstly, the policy gradient $\frac{1}{\alpha}\nabla_{\theta_j}J(\theta_j)$ transports particles towards regions of higher expected return, exactly as in the PG algorithms discussed previously. This term alone would result in the standard point estimate PG update rule in equation (3.1). However, since we are seeking a distribution over parameters, the kernel gradient term from SVGD $\nabla_{\theta_j}k(\theta_j,\theta_i)$ forces nearby particles to spread apart. From an optimisation perspective, this is an excellent mechanism for promoting exploration around regions of high return while still moving towards better policies. Finally, the prior gradient $\nabla_{\theta_j}\log q_0(\theta_j)$ pulls particles towards the mass of the prior distribution.

Much like general SVGD, with a single particle, the SVPG update rule reduces to a MAP estimate of $\exp(\frac{1}{\alpha}J(\theta))q_0(\theta)$. With an improper prior, $\nabla_{\theta_j}\log q_0(\theta_j) = 0$ resulting in standard gradient ascent.

An advantage of forming a distribution of parameters rather than converging to a single point estimate is that the distribution may locate **multimodal** solutions where different policy particles converge to different, locally optimal policies representing peaks in $q(\theta)$. These can occur in environments where fundamentally different behaviours produce similarly high rewards. SVPG is capable of finding these locally optimal policies within a single optimisation run. In contrast, finding multimodal solutions is often not possible, or at least requires greater discernment, with the other point estimate algorithms discussed in chapter 3. For example, reliably finding different optimal solutions to a multimodal environment with REINFORCE would require running multiple iterations with different initial configurations and simply hoping to find a unique solution.

Additionally, the kernel term $k(\theta_i,\theta_j)$ drives exploration while not sacrificing exploitation since different particles can explore different gradients while sharing the information they find. The attractive term $k(\theta_j,\theta_i)\nabla_{\theta_j}\log q(\theta_j)$ uses the kernel between particles to share information about the gradient. And the repulsive term $\nabla_{\theta_j}k(\theta_j,\theta_i)$ is what ensures particles are exploring different areas of the parameter space.

Chapter 5

Experiments

Now that we have a solid understanding of SVPG, we will use our own implementation of the algorithm to test several key aspects of the methodology that were not well described in Liu et al. (2017) and may have been overlooked. Our goal is not to provide benchmarks for fast learning in complex environments compared with other policy gradient algorithms, as was the goal in the original paper. Instead, we set out to analyse the interpretation of SVPG theoretically as a variational inference algorithm for approximating a Bayesian posterior, and the analytic properties that such an interpretation would entail. This chapter will look at the effect that using a weakly informative regularising prior has on learning, as well as how this effect changes if the prior encodes genuine prior information about high-return regions.

Since SVPG maintains a posterior distribution over policy parameters, we will study prior information as a distribution over θ. Generally speaking, there are two ways to encode prior information about network parameters into a reinforcement learning agent. The first is to initialise the policy with parameters sampled from the prior distribution and then proceed with standard optimisation. However, this approach means that as learning progresses, the original prior influence quickly diminishes, and the final policy is determined entirely by the optimisation objective and the observed data. As a result, this approach does not maintain any prior belief over the policy parameters and cannot be used for Bayesian inference. The second is to use an objective function, such as the update rule (4.6), that incorporates the prior distribution. In this case, the prior explicitly contributes to the gradient used to update the parameters of the policies.

An interesting observation of the SVPG paper is that they use a flat, improper prior with $\nabla_\theta \log q_0(\theta) = 0$ in all of their experiments, with no mention of the effect on learning or posterior approximation if an alternative prior is used. With a flat prior, the target posterior solely depends on the expected returns $q(\theta) \propto \exp(\frac{1}{\alpha} J(\theta))$. This means that particles will all be transported to regions of higher expected return, while the kernel term encourages exploration by repelling nearby particles and prevents all particles from collapsing to a single approximated optimum. Having an objective function that transports particles directly towards regions of high return without being pulled away by a prior may seem appealing for policy optimisation. However, encoding no prior information only shows that the algorithm acts as a diversity-encouraging optimisation algorithm

rather than an approximation of a Bayesian posterior.

Because of this, and in line with our goal, we will experiment with incorporating a selection of prior distributions in the SVPG algorithm to test if the learned particle distribution behaves consistently with the imposed prior. We wish to test to what extent the SVPG formulation acts in accordance with its interpretation as approximate Bayesian inference. As we discussed in the previous chapter, ideally, this would allow us to directly quantify our confidence in a learned policy, providing valuable information that can be used for high-risk problems.

5.1 Prior Predictive Check

We will begin by examining weakly informative, or regularising, prior distributions before inspecting the effect of including fully informative priors. In order to examine the effects that regularising prior distributions have on SVPG, we must first perform prior predictive checks, which tell us whether the policies induced by the prior cover a sufficiently broad range of the state space without an initial bias towards a specific solution. Starting from a collection of potentially interesting prior distributions, we will examine the behaviour of the policies that they induce. The distributions that we will investigate are given below. In each case, the components $\theta_1, \theta_2, \ldots, \theta_N$ of θ are independent and identically distributed.

- As a control, we will use the flat improper prior $q_0(\theta) = C$ used in Liu et al. (2017), which satisfies $\nabla_\theta \log q_0(\theta) = 0$. This results in no regularisation of the policy parameters. To simulate sampling from this "distribution", we sampled from a Gaussian with a standard deviation of 10^5. For our purposes, this gives an arbitrarily broad, symmetric and flat distribution.

- Gaussian Prior $q_0(\theta_i) = \mathcal{N}(0, \sigma^2)$. The joint distribution is

$$q_0(\theta) \propto \exp\left(-\frac{1}{2\sigma^2}\|\theta\|_2^2\right),$$

  hence $\log q_0(\theta) = -\frac{1}{2\sigma^2}||\theta||_2^2 + C$ and $\nabla_\theta \log q_0(\theta) = -\frac{1}{\sigma^2}\theta$. This prior induces L2-regularisation with a penalty term proportional to $||\theta||_2^2$. We will compare a narrow $\sigma = 0.1$ to a moderate $\sigma = 1$ and a broad $\sigma = 10$. These values were found to produce the most distinct behavioural patterns.

- Laplace prior $q_0(\theta_i) = \text{Laplace}(0, b)$. Here, the joint distribution is

$$q_0(\theta) \propto \exp(-\frac{\sum_i^N |\theta_i|}{b}).$$

  Therefore, $\log q_0(\theta) = -\frac{\sum_{i=1}^N |\theta_i|}{b} + C$ with $\nabla_\theta \log q_0(\theta) = -\frac{\text{Sign}(\theta)}{b}$. In the gradient, $\text{Sign}(\theta) = 1$ for all components $\theta_i > 0$ and $\text{Sign}(\theta) = -1$ for all components $\theta_i < 0$. At $\theta_i = 0$, the function is not differentiable, so we define $\nabla_\theta \log q_0(\theta) = 0$. The L_1 regularisation from Laplace priors, with penalty term proportional to $||\theta||_1$, differs from L2-regularisation since it

shrinks parameters linearly and, unlike Gaussian, allows exact zeros to be optimal. This encourages sparsity in the parameters of optimal solutions. The variance of the Laplace distribution is $2b^2$ hence a reasonable selection of hyperparameters to investigate based on the selection for the Gaussian prior are $b = \{\frac{0.1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{10}{\sqrt{2}}\}$. We found that the narrow and broad Laplace distributions gave similarly identical or deterministic behaviour as the corresponding Gaussians, and for this reason we only include the plot for $b = \frac{1}{\sqrt{2}}$.

- Cauchy prior $q_0(\theta_i) = \text{Cauchy}(0, \gamma)$ with joint distribution

$$q_0(\theta) \propto \prod_{i=1}^{N} \frac{1}{1 + (\frac{\theta_i}{\gamma})^2}.$$

The log density is $\log q_0(\theta) = -\sum_{i=1}^{N} \log(1 + (\frac{\theta_i}{\gamma})^2) + C$ with gradient $\nabla_\theta \log q_0(\theta) = -\frac{2\theta/\gamma^2}{1+(\frac{\theta}{\gamma})^2} = -\frac{2\theta}{\gamma^2+\theta^2}$. This prior was included in addition to the previous two because it has much heavier tails, meaning extreme values are more probable. In terms of regularisation, the penalty term is proportional to $\sum_{i=1}^{N} \log(1+(\frac{\theta_i}{\gamma})^2)$, which grows much more slowly than L1 and L2 penalties. This means the force pulling large θ towards the centre will be weaker. Interestingly, the Cauchy distribution has no well-defined variance, so we experimented with $\gamma \in [0.05, 0.1, 1]$ and found that $\gamma = 0.05$ was too narrow and $\gamma = 1$ was too broad for diverse, stochastic behaviour with no initial bias.

The general methodology for performing a PPC (Stan Development Team, 2024; Gabry et al., 2019) is to sample parameters from the prior distributions $\theta \sim q_0(\theta)$ and then simulate data $\mathcal{D}^{\text{sim}} \sim p(\mathcal{D}|\theta)$ from a likelihood function using these sampled parameters. The resulting distribution

$$p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)\ d\theta$$

is called the **prior predictive distribution** and represents the distribution of data we would expect to observe if the parameters were distributed according to our prior beliefs. Comparing samples $\mathcal{D}^{\text{sim}} \sim p(\mathcal{D})$ from this distribution with plausible outcomes tells us whether the prior is encoding useful information.

In our case, the prior is a distribution over parameters of a neural network, which is not directly interpretable. Because of this, examining the behaviour of the policies induced by these parameters is more meaningful than simply examining the distributions of the parameters. The data $\mathcal{D}$ that we are simulating with our sampled θ represents sampled trajectories of the environment while acting under policy π_θ. We would like to ensure that these trajectories correspond to a sufficiently diverse and reasonable set of behaviours by analysing the state visitation and the distribution of actions taken in a given state.

Our aim is to identify priors that give us a diverse set of policies, each of which being diverse in behaviour. This will ensure that our prior does not encourage overly confident, deterministic behaviour and that the prior distribution covers

a reasonably wide variety of policies. This is desirable for the following reason. If our prior distribution leads to similar or highly deterministic policies, then it is already imposing a strong bias towards certain behaviours before any data is observed, which is not the purpose of a weakly informative prior. A suitable prior should not induce deterministic behaviour everywhere, as this may exclude high-return regions and does not cover a wide range of plausible behaviours.

Most environments have more than two state dimensions, making it challenging to view in a single plot. One notable exception is the MountainCar (Farama Foundation, 2025*b*) environment, however, the reward sparsity makes it challenging for simple policy gradient methods to solve. For our experiments, we will use an environment with denser reward feedback to clearly compare the relative quality of different suboptimal policies. A suitably simple environment is CartPole (Farama Foundation, 2024*a*), which has 4 state dimensions, the cart's position and velocity, as well as the pole's angle and velocity. We can project this onto a two-dimensional subspace containing the cart's position and pole angle without losing too much interpretability. Additionally, the action space is binary with $a = 0$ corresponding to a left movement and $a = 1$ corresponding to a right movement.

There are two key considerations we made when deciding what information to display. Firstly, we would like to observe diversity among trajectories for a given sampled policy as well as diversity between sampled policies. Examining both quantities in a single plot is challenging. Using separate plots ensures that we do not mistakenly assess a given prior as suitable when it produces nearly identical policies that return random actions, or a diverse set of policies that all produce deterministic actions in different directions.

An additional consideration is that the chosen prior causes policies to visit different regions of the state space as well as to behave differently within the same state. This means that if we compute metrics about policies in a state using only the policies that reached it, we will get misleading information about the diversity of policies in that region. This is because the value we compute in that case, conflates both state occupancy induced by the prior and the action choices of policies induced by the prior. In line with both of these considerations, we will proceed as follows.

We will collect $M = 1000$ sampled policies from a given prior distribution, and then collect $n = 10$ trajectories under each policy. We chose $M \gg n$ because the randomness from the former relates to the prior distribution that we wish to check, whereas the latter is only dependent on the starting state distribution ρ_0 and the stochasticity of the policy. We found that $n = 10$ is sufficient to provide a good representation of the policy dynamics, and $M = 1000$ provides a good coverage of the prior. We expect the returns to be similarly poor amongst priors because none of the policies has undergone any training. Therefore, we will look specifically at the states and actions of each policy along these trajectories. We will discretise the projected state space into a grid of 45×45 cells and consider the cells visited by at least one policy. Within these visited cells, we will randomly select a set of 10 visited test states (or all visited states in that cell if there are fewer than 10). In each test state across every cell, we will record two metrics described below, and average these values over the number of sampled states in

the cell, resulting in two distinct heatmaps for each prior.

Firstly, we will compute the mean action entropy in that state across all policies,

$$\frac{1}{M}\sum_{i=1}^{M} -p_i \log p_i - (1-p_i)\log(1-p_i),$$

where $p_i \in [0,1]$ represents the probability of policy i moving right in the current state. This will tell us how stochastic the policies sampled from our prior are. If all of the policies deterministically select right in a given cell, the entropy is minimised at 0 and likewise if they all deterministically select left. If every policy is maximally random with $p_i = 0.5,\ \forall i$, the mean entropy is maximised at $\log 2 \approx 0.69$. The reason we selected entropy rather than simply averaging p_i, is that if half of the policies deterministically select left and the other half deterministically select right, a plot of average p_i would display 0.5 which is indistinguishable from a collection of identical and random policies. On the other hand, the mean entropy would still be zero, preserving the interpretation of low entropy representing more deterministic policies.

We will also compute the variance $Var(\boldsymbol{p}),\ \boldsymbol{p} = p_{1:M}$ of actions selected across sampled policies in that state. Clearly this variance is bounded below by 0 when all policies are identical and is bounded above by

$$Var(p_i) = \mathbb{E}[p_i^2] - \mathbb{E}[p_i]^2 \leq \mathbb{E}[p_i] - \mathbb{E}[p_i]^2 = \mathbb{E}[p_i](1-\mathbb{E}[p_i]) \leq \frac{1}{4}.$$

We used the fact that $0 \leq p_i \leq 1 \implies \mathbb{E}[p_i^2] \leq \mathbb{E}[p_i]$. This metric gives us a clear indication of how much different policies disagree about what action to take in a certain cell.

Finally, the policy network we use for these experiments has two hidden layers with 64 units each and ReLU activation functions. The output layer uses a softmax activation function as discussed in Chapter 3.

5.1.1 Results

The values of these two metrics averaged over each discretised cell for each prior distribution are shown in Table 5.1. The heatmaps displaying state visitation and the location of the recorded values are shown below using a logarithmic scale. We will now discuss the findings from them.

The approximately improper prior produces unrestricted arbitrarily large weights, and we can see from Figure 5.1 that this lack of restriction results in entirely deterministic behaviour at each state. The behaviour between policies is incredibly diverse, as expected, but the determinism imposed by each policy means that this diversity comes from different policies with maximum confidence in certain actions rather than genuine stochasticity in the policies. A similar effect can be seen in Figure 5.4 representing $q_0(\theta) = \mathcal{N}(0, 10^2)$. While the plots are almost identical, Table 5.1 shows us that the Gaussian leads to a small amount of action entropy, showing that policies are not entirely deterministic. Even so, this shows us that a Gaussian with a standard deviation of 10 produces behaviour very similar to the

uninformative prior for this particular environment and network architecture.

On the other hand, the narrow Gaussian prior in Figure 5.2 represents a collection of policies which act almost identically. Even though the actions they produce are maximally stochastic, the restriction to an identical policy means this prior does not suitably explore the state space or provide a diverse set of policies.

The final three policies shown in Figures 5.3, 5.5 and 5.6, display the behaviour of $q_0 = \mathcal{N}(0, 1)$, Laplace$(0, \sqrt{2})$ and Cauchy$(0, 0.1)$ respectively. They each provide a diverse set of policies, as seen by the high variance in action probabilities between policies. Additionally, the policies contain a meaningful amount of entropy, indicating they do not all collapse to confident, deterministic policies as the first two broad priors did. For this reason, these are the priors that we will use in the following experiment.

Prior Distribution	Mean Policy Entropy	Mean Variance
Improper	0.000	0.220
$\mathcal{N}(0, 0.1^2)$	0.687	0.003
$\mathcal{N}(0, 1)$	0.094	0.201
$\mathcal{N}(0, 10^2)$	0.001	0.217
Laplace$(0, \sqrt{2})$	0.099	0.199
Cauchy$(0, 0.1)$	0.154	0.185

Table 5.1: Table displaying the average action entropy of sampled policies and the average action variance between sampled policies.

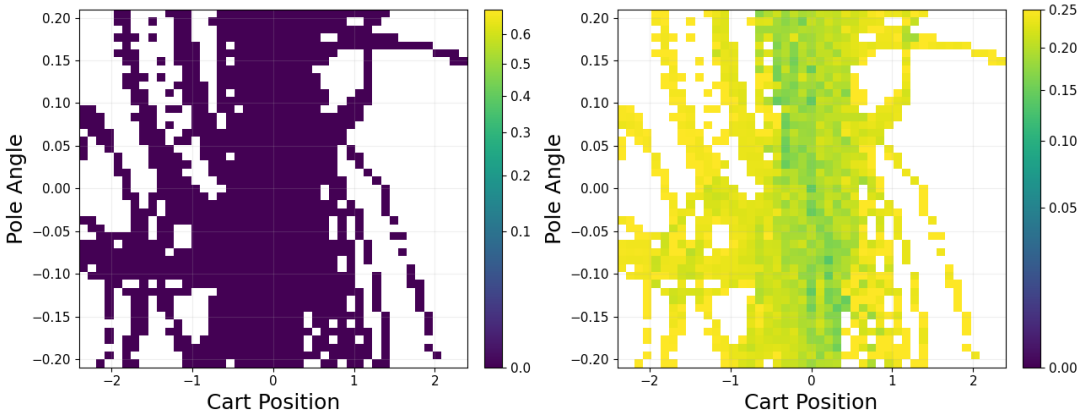

Figure 5.1: PPC for an improper prior. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

5.2 Prior Performance

Now that we have observed the typical behaviour of policies induced by our set of priors, we will study the effect of incorporating these priors into the SVPG

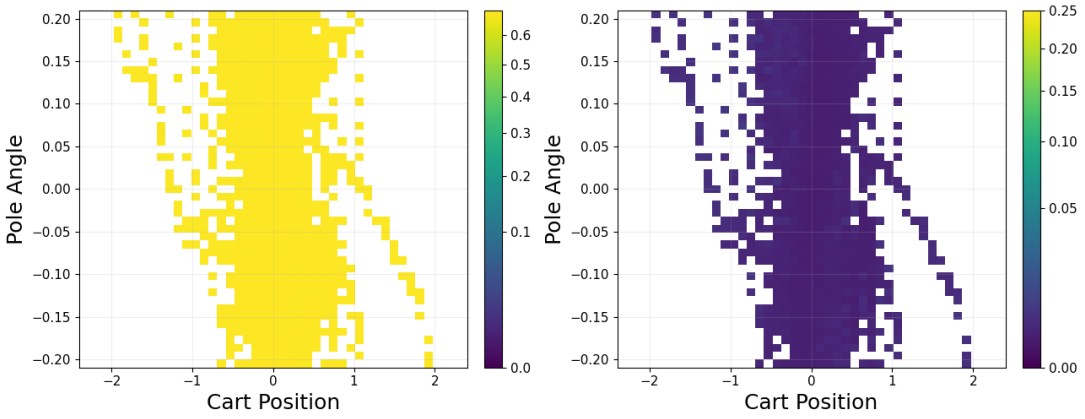

Figure 5.2: PPC for $\mathcal{N}(0, 0.1^2)$. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

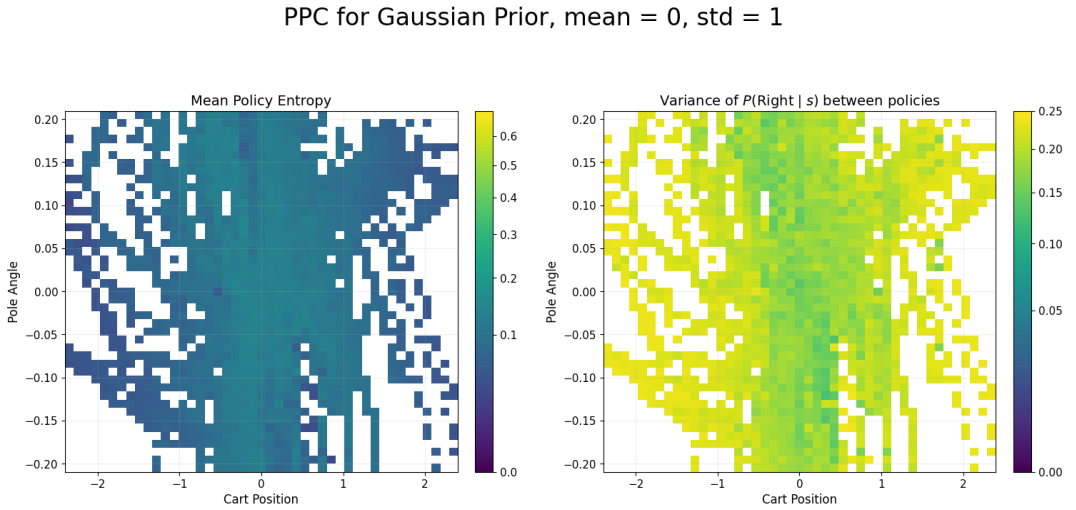

Figure 5.3: PPC for $\mathcal{N}(0, 1)$. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

update rule on the learning dynamics. For this, we will begin by explaining our choice of the remaining components of the algorithm, which must be fixed.

Firstly, our choice of the policy gradient estimator requires some discussion. The update (4.6) depends directly on this estimate, so computing a low-variance estimator is essential to form a meaningful posterior. With the competing forces from the policy gradient, prior gradient, and kernel repulsion, reducing the noise in the estimates of the "likelihood" term is particularly important. As we have discussed in Section 3.3.1, this can be achieved by extending the REINFORCE algorithm to batched samples of trajectories as well as subtracting a baseline function from the Monte Carlo estimates of the returns G_t. We demonstrated that these modifications produce a meaningful reduction in estimator noise.

The alternative that we explored is to reduce variance further at the cost of introducing bias by learning a value function $V_\psi(s) \approx V^\pi(s)$ via a critic, and

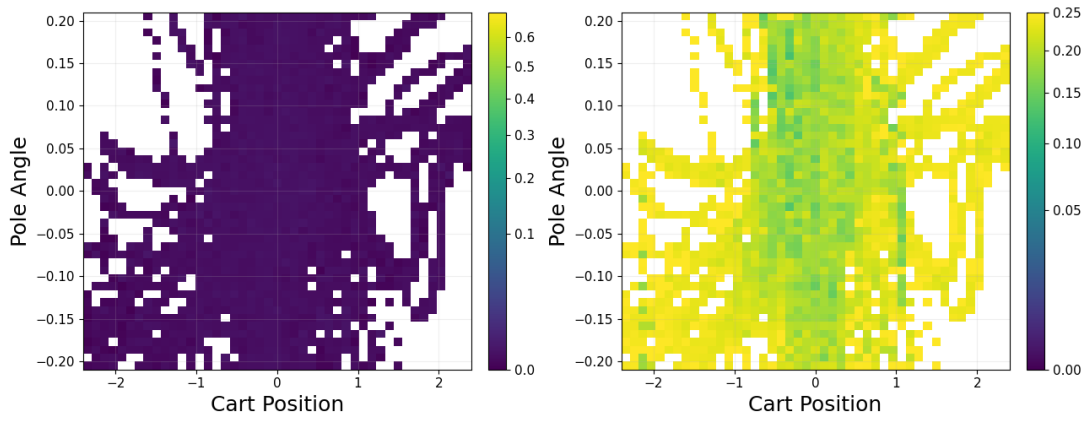

Figure 5.4: PPC for $\mathcal{N}(0, 10^2)$. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

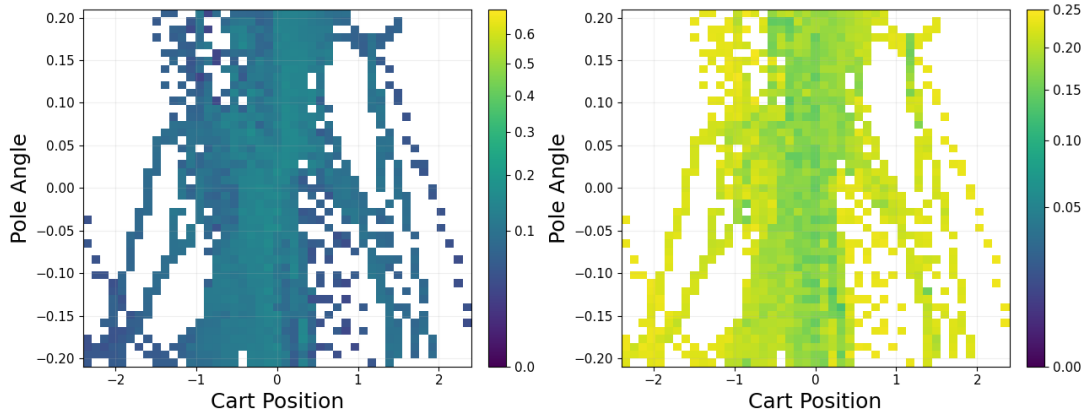

Figure 5.5: PPC for Laplace$(0, \sqrt{2})$. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

then using bootstrapped updates to compute residuals $\delta_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t)$ which are then used to estimate the advantage function in GAE via $\hat{A}_t = \sum_{l=0}^{\infty}(\gamma\lambda)^l\delta_{t+l}$. Using $\hat{A}_t$ rather than Monte Carlo returns when computing the policy gradient has been shown to significantly reduce variance however unless $\lambda = 1$, it is biased due to the bootstrapped value function predictions.

For standard RL, where we are interested exclusively in maximising the objective, introducing some bias in this way can often aid the learning process and improve stability due to the relatively low variance. However, when interested in forming a posterior distribution, as in SVPG, if our estimator is biased and $\mathbb{E}[\widehat{\nabla_\theta J(\theta)}] \neq \nabla_\theta J(\theta)$, the algorithm will no longer, in general, produce samples from a well defined posterior distribution which is what we are seeking from a variational inference method. If our biased estimator has expectation $\mathbb{E}[\widehat{\nabla_\theta J(\theta)}] = g(\theta) = \nabla_\theta \tilde{J}(\theta)$ for some $\tilde{J}(\theta) \neq J(\theta)$, then we could interpret the SVPG algorithm as producing samples from a distribution $\tilde{q}(\theta) \propto \exp(\frac{1}{\alpha}\tilde{J}(\theta))q_0(\theta)$. However, this assumption is not true in general, as we have no guarantee that the expectation

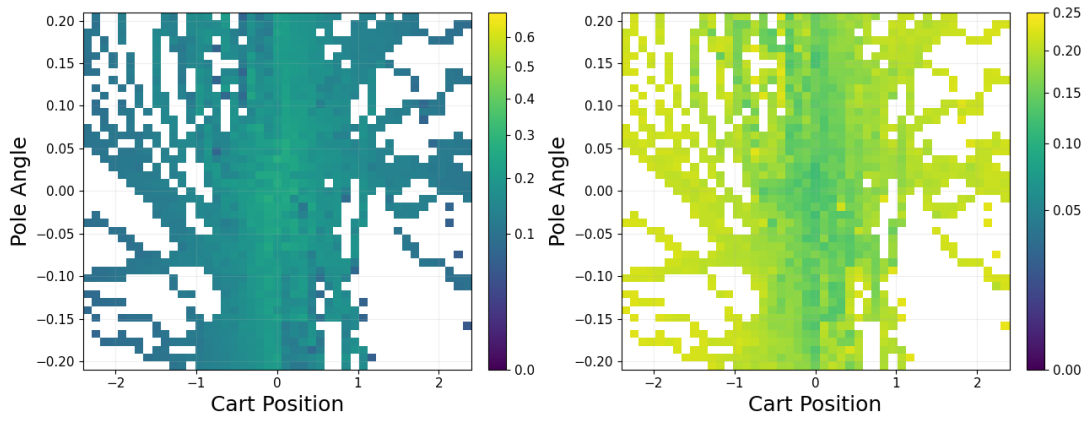

Figure 5.6: PPC for Cauchy$(0, 0.1)$. **Left** shows the mean policy entropy in each cell. **Right** shows the variance of action probabilities across policies.

of such a biased estimator can be expressed as a scalar gradient. Because of this, using a biased estimator for the policy gradient means the theoretical properties of SVPG as a variational inference method can no longer be interpreted directly.

This does not mean the algorithm can only be used with unbiased PG estimators. In fact, the paper uses a biased estimator and demonstrates high returns, which again highlights the author's intention to use the algorithm as an optimisation method rather than a VI method. Additionally, we will use a biased estimator in Section 5.3 when we are more interested in investigating the learning effect of prior information than the Bayesian interpretation. To attempt to preserve the theoretical properties, we will use a REINFORCE policy gradient estimator with 5 sampled trajectories per update and a constant baseline. This formulation is unbiased, and we found that it performed well in our experimentation in Section 3.3.2.

Most important for examining prior influence is our choice of α. Recall the target distribution (4.5) tells us that as $\alpha \to \infty$, the posterior distribution would more closely resemble the prior. While an $\alpha \to 0$ means that samples from $q(\theta)$ will be distributed around regions of high expected return. When variational inference is introduced to approximate this target distribution, giving us (4.6), α gains an additional role of controlling the magnitude of the target distribution. With $\alpha \to \infty$, the algorithm acts as standard SVGD for approximating $q_0(\theta)$, and $\alpha \to 0$ weights the likelihood component of the target distribution highly, making the repulsive term negligible in the gradient and causing the particles to move in the direction of higher expected return with negligible particle interaction. The magnitude of the overall gradient is absorbed into the learning rate. Our goal is to determine whether changes in prior distribution and α produce changes in learning dynamics that are consistent with the choice of prior and temperature we use.

Since we are not investigating the choice of kernel function or particle interaction specifically, we found that the key result of our experiment is already visible with a single particle. Here, the effect of the prior can be isolated from the influence of the kernel term, providing clearer and more interpretable plots.

For this reason, and due to the length constraints of this dissertation, we restrict our analysis to the single-particle case.

5.2.1 MAP estimation

With one particle, the kernel gradient is zero, meaning the temperature α has only the role of weighting policy gradient and prior gradient influence (assuming the learning rate is adjusted according to the magnitude of the resulting gradient). We execute the SVPG algorithm using the priors that our PPC determined to be suitably weakly informative with a range of $\alpha \in \{1.0, 0.1, 0.03\}$ to observe the result of decreasing the prior influence on the gradient. Note that the values of α that we are examining are vastly different to the $\alpha = 10$ used by Liu et al. (2017). This is because, with an improper prior, the only consideration one needs to make when calibrating α is the tradeoff between the exploitation of the attractive term and the exploration of the repulsive term. Our results demonstrate why no α larger than 1 was discussed.

Figure 5.7: CartPole learning curves representing MAP estimation using different prior distributions with $\alpha = 1.0$.

We see that in all three of the values of α we used, using a regularising prior distribution led to much weaker convergence and lower final returns than with an uninformative prior. With $\alpha = 1$, the learning curve in Figure 5.7 shows us that only the improper prior is able to learn, suggesting the regularisation strength of the priors is too strong for the particle to overcome. Decreasing α lowers this regularisation strength, and with $\alpha = 0.1$, shown in Figure 5.8, we see that the particle regularised by the Gaussian prior is able to improve marginally, although the regularisation effect is still clearly dominating as the particle moves further away from the mass of the prior. We also observe an unusual pattern of the curve rising and falling well below the maximum returns, which will be explored in the following chapter. Finally, as we decrease α to 0.03, we see in Figure 5.9 that

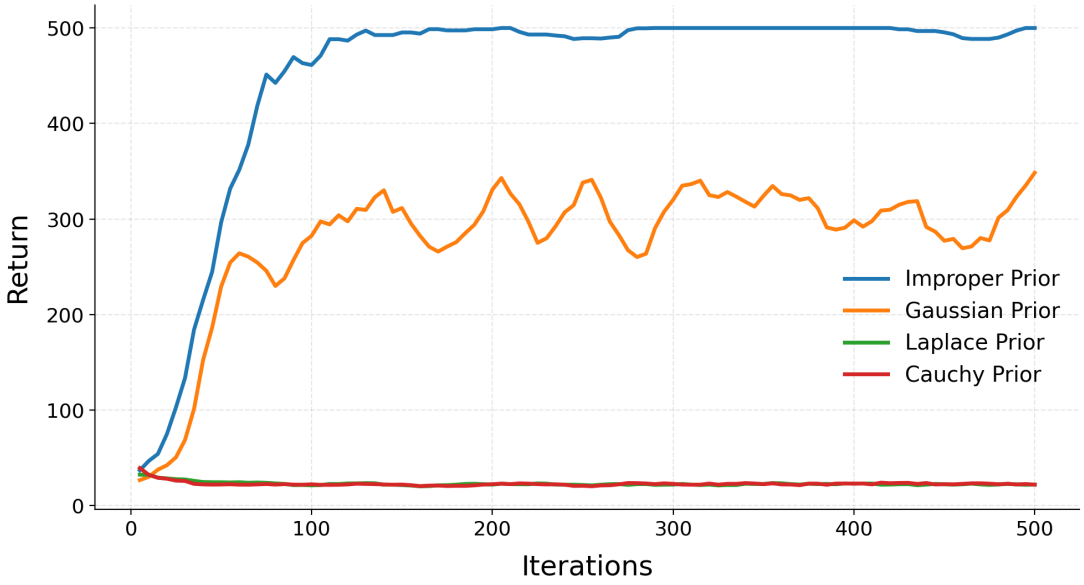

Figure 5.8: CartPole learning curves representing MAP estimation using different prior distributions with $\alpha = 0.1$.

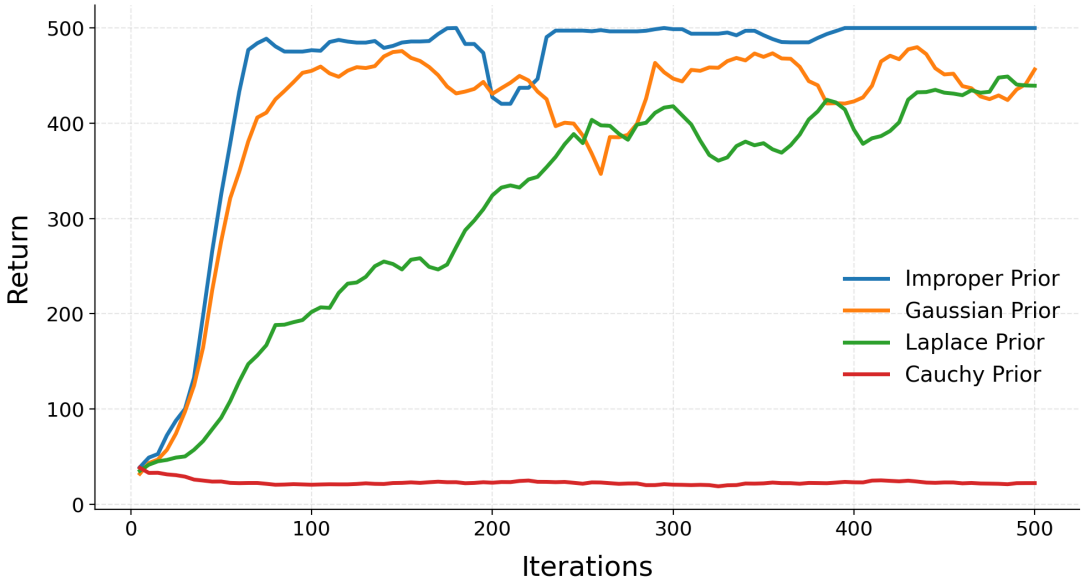

Figure 5.9: CartPole learning curves representing MAP estimation using different prior distributions with $\alpha = 0.03$.

both the Laplace regularised particle as well as the Gaussian regularised particle begin to learn. The rising and falling pattern is still present in the Gaussian curve, although it is less distinct, and notably closer to the maximum return. Additionally, the strength induced by the Cauchy prior appears to still be too strong for the final particle to overcome.

To understand these behaviours more precisely, we then examined how $||\theta||_2$ and the magnitudes of the weighted policy gradient and prior gradient terms change during training. This allows us to determine whether the weaker learning under regularising priors is simply due to particles remaining in a region near zero, or whether the priors are still exerting a strong force as the particles approach regions with higher returns. For brevity, we will not include all of the resulting plots, as many display recurring ideas. The most insightful diagnostic graphs show the evolution of the norm of θ, shown in Figure 5.10, and the ratio $\frac{||\nabla_\theta \log q_0(\theta)||_2}{||\frac{1}{\alpha}\nabla_\theta J(\theta)||+||\nabla_\theta \log q_0(\theta)||}$ displaying prior dominance in the update rule shown in Figure 5.11. We chose to display both of these plots for $\alpha = 0.03$ as it is at this value that multiple regularised particles begin to move from the origin.

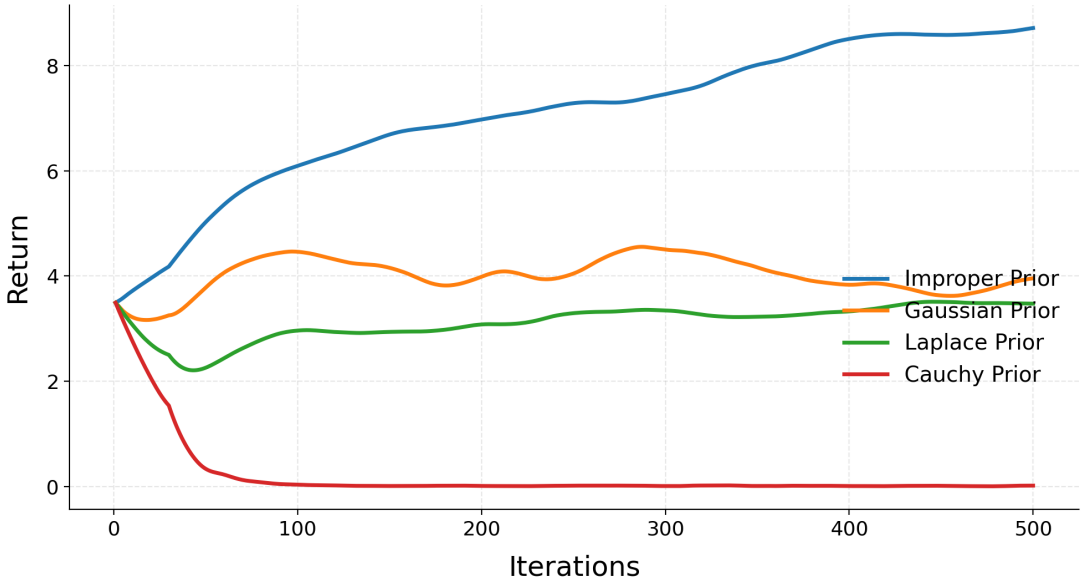

Figure 5.10: The evolution of $||\theta||_2$ for different prior distributions with $\alpha = 0.03$.

The norm of the parameter vector broadly tracks learning performance. For the priors and values of α where the regularisation prevents learning, the parameter norm immediately shrinks to zero. When α is reduced to 0.03, the Gaussian and Laplace priors are able to maintain much larger parameter norms, which aligns with the improved returns seen in the learning curves. This shows us that the particles are unable to find regions of high return near the mass of the prior, and improved learning only comes when the prior is made weak enough to allow parameter growth. However, this diagnostic alone does not show whether the prior influence has disappeared, and only shows that weaker regularisation allows particles to move further from the origin.

Figure 5.11 displays the relative prior influence compared to the weighted policy gradient influence. Clearly, for the improper prior, the prior fraction remains

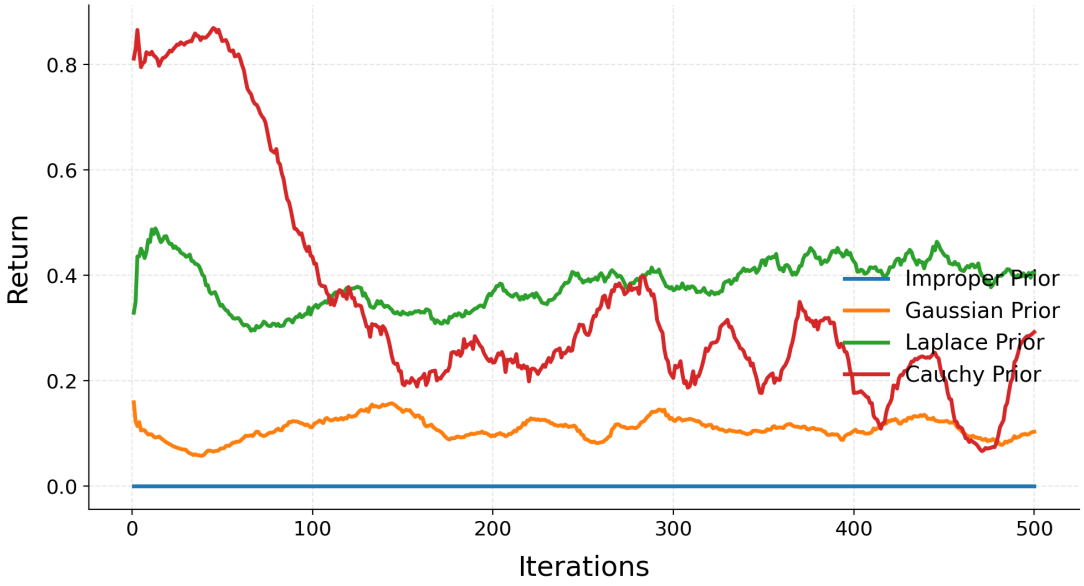

Figure 5.11: The evolution of the prior dominance fraction for different prior distributions with $\alpha = 0.03$.

identically zero throughout training since it has no gradient term. We can see that both the Laplace and Gaussian prior influences remain high as θ moves away from the origin. This shows that even when these particles are able to learn, the prior term is still contributing a large proportion of the update. In contrast, the Cauchy prior also has a fairly large prior fraction, but further inspection of the policy gradient norm shows that the weighted policy gradient remains too small for the particle to overcome the prior gradient and make notable improvements.

5.3 Genuine Prior Information

The previous experiments show that using prior distributions purely for regularisation appears to make learning much more challenging due to the prior gradient term, which remains large, pulling particles away from regions of high return as they move further from the mass of the prior. Unlike standard Bayesian learning, where the likelihood function becomes more sharply peaked as we collect more data, the policy gradient decreases in magnitude as the particles approach the local optimum. This causes the prior term to dominate, and learning becomes challenging.

An alternative motivation for using a prior, which we mentioned in Section 4.2.1, is when we have genuine prior information about the location of regions of high return. To investigate how SVPG responds to this type of prior, we first produce an approximation of the optimal parameter vector θ^* by training a policy on Lunar Lander (Farama Foundation, 2024*b*) using the GAE estimator, with $\lambda = 0.95$ and 5 trajectories per parameter update. The final policy was evaluated over 2000 episodes using greedy evaluation and achieved returns of 259.14 with a standard deviation of 51.15. According to the documentation, consistent

evaluation returns above 200 represent a successful policy, so we determine this to be suitable for our purposes.

We then use the saved weights $\widehat{\theta}$ of this policy to define an informative prior $\mathcal{N}(\widehat{\theta}, 1)$ centred around a region we know to be of high expected return. We then compare the learning of four sets of particles with unique configurations updated using SVPG to determine if the SVPG prior term has a positive effect on learning in the case that it encodes genuine information about regions of high expected return. We wish to find out if encoding this prior information into the update rule results in more stable and successful learning than simply using it directly for initialisation. Our experiment will compare the learning curves of the following four configurations:

- A control with standard random initialisation and an improper prior,
- random initialisation with an informative prior $q_0(\theta) = \mathcal{N}(\widehat{\theta}, 1)$,
- initialisation around $\widehat{\theta}$ with an improper prior,
- initialisation around $\widehat{\theta}$ with the informative prior $q_0(\theta) = \mathcal{N}(\widehat{\theta}, 1)$.

This experiment allows us to study the two different effects of prior information on both initialisation and as a persistent gradient term. Since we are now more interested in the impact on learning that prior information has on SVPG rather than the theoretical Bayesian interpretation, we will use the lower variance GAE estimator in each of the above configurations and $M = 1$ particle. The result of this experiment is shown in Figure 5.12.

Figure 5.12: The learning curves of four different configurations of informative prior information in Lunar Lander.

These results demonstrate that an informative prior is practically useful in the SVPG formulation. We can see that particles initialised in a successful region have

high returns initially. However, if no prior term is included in the update rule, the noise from policy gradient estimates can cause significant dips in performance. Since the ADAM optimiser uses gradient information accumulated from past iterations (Kingma and Ba, 2014), these dips in learning set back progress to an extent that is difficult to recover from. On the other hand, the prior acts as a constant force towards the high-return region, meaning that as particles explore around this region, if they move towards a less optimal region, the prior term will draw them back, dramatically improving stability. This phenomenon relates directly to the non-standard form of the likelihood function in the Bayesian interpretation of SVPG. Since the effect of the prior does not diminish with additional data, this pull towards the more optimal region will effectively guide the particles even after many training iterations.

Chapter 6

Results and Discussion

In this chapter, we provide interpretations of the results found in the previous experiments in the context of the Bayesian interpretation of SVPG, as well as outlining potential limitations to these results. As a consequence of these interpretations, we illustrate several critiques of this interpretation that prevent the algorithm from being an effective variational inference method that Chapter 4 motivated the desire for. Finally, we propose alternative interpretations of the formulation of the target distribution as a Boltzmann distribution and, separately, using the framework of general Bayesian inference.

6.1 Results

The results of the experiments that we presented suggest that SVPG should be interpreted less as a standard Bayesian posterior inference algorithm over policy parameters, and more as an optimisation method for training a set of policies which interact with each other to promote exploration. We discovered that when a prior is not encoding genuine information, it acts as a persistent force that is not dominated by additional trajectory data. Since policy gradient estimates have very high variance, the prior gradient term can often have the greatest influence on parameter updates, and when the prior is acting as a regulariser, this influence can often worsen the policy. If the temperature is sufficiently low or the prior sufficiently broad, we saw that the particles may be able to move to a region of higher return. However, as they approach a local optimum, the true policy gradient term required to further improve the policy becomes increasingly small. This results in the prior gradient term having a greater relative effect on the particle's updates and causing them to leave the high-return region. Since the additional data collected is not included in the formulation of the target distribution, as in ordinary Bayesian inference, this effect will prevent the particles from being able to stably remain in a high-return region if that region lies away from the mass of the prior.

We found that the effect of the prior on learning depends not only on the scale of the prior and the behaviour of the policies that it induces, but also on the prior's shape due to the different ways $\nabla_\theta \log q_0(\theta)$ can depend on θ. For a Gaussian prior, $|\nabla_\theta \log q_0(\theta)| = \frac{|\theta|}{\sigma^2}$, meaning the prior effect increases with the

size of θ. We saw explicitly in Figure 5.8 that this led to an oscillatory curve. As the particle increases its return and moves to different areas of the parameter space, the prior gradient grows in magnitude. This pulls the particle back and leads to a large decline until the parameters are shrunk enough for $|\nabla_\theta \log q_0(\theta)|$ to be sufficiently small to stop hindering learning.

In contrast, a Laplace prior has constant log gradient magnitude $|\nabla_\theta \log q_0(\theta)| = \frac{1}{b}$, meaning the strength of the prior does not increase as θ moves further from the origin. Therefore, for θ close to the origin, where the strength of the Gaussian prior is low, particles regularised by the Laplace prior must overcome this constant force. This explains why the Laplace prior was unable to learn in our experiment when $\alpha = 0.1$. As α was decreased, the policy gradient term became large enough to overpower the constant pull of the Laplace prior, resulting in slow learning that did not display the same oscillatory behaviour as the Gaussian prior. The diagnostic plot in Figure 5.11 shows us that the constant prior term plays a persistent and significant role in the updates of the particle.

Finally, the Cauchy prior satisfies $|\nabla_\theta \log q_0(\theta)| = |\frac{2\theta}{\gamma^2+\theta^2}|$, which grows rapidly with $|\theta|$ until $|\theta| = \gamma$, at which point the gradient decreases. This rapid growth for small θ is what appears to prevent the particle from learning successfully, since any growth of θ is quickly reversed by the increasing prior gradient. The two diagnostic plots show that the particle was quickly shrunk to zero at initialisation, after which the prior fraction remained significant. We see that the fraction exhibits frequent spikes, which likely represent small improvements in the policy being counteracted.

The final experiments compared the effects of informative prior information as a method for particle initialisation, as well as in the SVPG prior distribution. This showed that in the Lunar Lander environment, an informative prior centred at a region of high expected return can be practically useful in SVPG. Particles initialised in this region had high initial performance, but if no prior term was included, the noise from the policy gradient estimates led to significant dips in performance. In contrast, the informative prior acted as a persistent force pulling particles back to the high return regions and producing dramatically more stable learning.

6.1.1 Limitations of the Experiments

One noteworthy limitation is that our experiments concerning the regularisation effects of prior distributions in SVPG were conducted exclusively on the CartPole environment. This was a decision made to be consistent with the results of our PPCs, where the environment was chosen to be sufficiently low-dimensional for the PPC plots to be interpretable, while also having sufficiently dense rewards to demonstrate the effects of weaker learning methods. While this decision was made with care, it does limit the generality of the results we observed. However, the conceptual issue that the prior is not dominated by the likelihood with additional data is a consequence of the SVPG formulation itself and not the environment used. The same limitation applies to our claim regarding the effects of informative prior distributions. However, the reasoning of the informative prior providing a persistent pull towards regions of high return remains true regardless of the

environment.

Secondly, many results we observed were directly related to the hyperparameters of the prior used, controlling its strength. This was a decision made as a result of our PPC experiments and our desire to select a weakly informative prior which produces diverse behaviour and does not collapse to a maximally confident deterministic policy. In hindsight, a broader prior would exhibit a weaker force pulling particles towards the origin and therefore have less of a negative effect on learning. However, the result that we discovered arose from following standard Bayesian inference techniques and analysing the behaviour of policies sampled from the prior.

Finally, we only presented the experiments produced using a single policy particle to produce a MAP estimate. As we explained in the previous chapter, the reason for this was that including kernel interactions would make the influence of the prior more difficult to isolate and interpret. However, it is worth noting this as a limitation since one of the key features of SVPG is the collection of interacting particles.

6.2 Issue with the Bayesian Formulation

By analysing the posterior formulation in SVPG and comparing it with standard Bayesian inference, we discovered several key issues with the ordinary Bayesian interpretation of the algorithm. While SVPG as an optimisation method has been shown to outperform simple policy gradient algorithms under suitable conditions, the problems described below render SVPG practically inapplicable as a method of Bayesian inference to solve the problems discussed in Chapter 4.

6.2.1 Finite-Particle and Gradient Approximation

Firstly, SVGD (Liu and Wang, 2016), which forms the update rule that SVPG uses to transport policy particles towards the approximate posterior, guarantees convergence under suitable conditions as the number of particles tends to infinity (Salim et al., 2022). In practice, SVPG represents each particle as its own policy, and each policy must collect its own trajectory samples to estimate its individual policy gradient. This means that using a large number of particles is computationally infeasible, and in practice, one typically uses well below 50 particles. The resulting empirical particle distribution is therefore only a very coarse approximation to the ideal SVPG target distribution. Additionally, since θ represents a very high-dimensional vector, the structure of this target distribution is very difficult to represent with a small number of particles. It is also the high dimensionality of the particles that makes it challenging to visualise the empirical posterior distribution in a meaningful and interpretable way.

The repulsion from the kernel is intended to encourage exploration of multiple posterior modes to prevent collapse to the MAP. However, with only a small number of particles, the diversity that can be maintained in this way is very limited. This results in a kernel term which acts less like a mechanism to represent a complex posterior distribution and more like an exploration-inducing interaction

term between a small number of particles. This is exactly what Liu et al. (2017) desired when developing an exploration-driven optimisation algorithm, but it dramatically weakens the interpretation of the particles as an approximation to a Bayesian posterior distribution. Recent work by Das and Nagaraj (2023) studies the theoretical properties of finite-particle variants of SVGD. They discuss the weak convergence properties of SVGD when using a finite number of particles and propose alternative algorithms with provably fast finite-particle convergence rates.

There are, therefore, two approximations occurring simultaneously in SVPG. Firstly, the empirical particle distribution is only a finite, coarse approximation to the target posterior. Secondly, the score term $\nabla_\theta \log p(\theta|\mathcal{D})$, used in SVGD to move towards the target distribution, is only an approximation in SVPG. This is due to the recurring theme of $\nabla_\theta J(\theta)$ being an object requiring estimation through trajectory sampling. We discussed in the previous chapter that an unbiased estimator is required to preserve the true update direction in expectation. However, the resulting estimator still has high variance, making the parameter updates unstable.

The convergence guarantees for SVGD also assume access to the exact score of the target distribution. However, SVPG replaces this score with a high-variance PG estimate. As a result, the algorithm is not only approximating the posterior with finitely many particles, but is also transporting those particles using an approximation of the correct update direction.

The consequence of these two forms of approximation is that even if the target posterior were valid in a strict Bayesian sense, the approximation of it in this way would be too weak in practice to produce informed uncertainty quantification.

6.2.2 Absence of a True Fixed Likelihood

The formal definition of a likelihood function (Wikipedia contributors, 2026*b*) is as follows. If $x \mapsto f(x|\theta)$ is a probability density or mass function for fixed θ, where x is a realisation of the observable random variable X, then for observed data x, the likelihood function is

$$\theta \mapsto f(x|\theta),$$

viewed as a function of θ with x fixed.

However, in SVPG, the term $\exp(\frac{1}{\alpha}J(\theta))$ acts as the likelihood function despite not being derived from a probability distribution for fixed observed data. Instead, it is computed using the expected return under the current policy, meaning there is no fixed dataset x for which this term can be interpreted as a true likelihood function in the ordinary Bayesian sense.

Additionally, our estimates of $J(\theta)$, or more precisely $\nabla_\theta J(\theta)$, rely directly on the current policy θ. Since θ is the variable that we update in the SVPG algorithm, there is no fixed distribution of the 'data'. This distinction is one of the strongest motivators to adopt a general Bayesian interpretation discussed in Section 6.3.2.

6.2.3 Persistence of the Prior

Our final critique of the ordinary Bayesian interpretation of the SVPG objective reiterates the result that we found in our experiments. In standard Bayesian inference, the log posterior is expressed

$$\log p(\theta|x_{1:N}) = \sum_{i=1}^{N} \log p(x_i|\theta) + \log p_0(\theta),$$

for i.i.d observations x_i. We studied this formulation in ridge regression in Section 4.2.1. The relevant observation is that as we collect more data and increase N, the contribution of the likelihood increases while the prior contribution remains constant. This results in the effect of the prior distribution on the posterior diminishing. In SVPG, the expression of the log likelihood $\log p(\mathcal{D}|\theta) = \frac{1}{\alpha}J(\theta)$ is independent of the size of our data, meaning the prior contribution persists throughout training. We saw in our experiments in Section 5.2.1 that this issue prevents the standard use of prior distributions as regularisers.

6.3 Alternative Interpretations

The critiques we discussed in the previous section strongly suggest the ordinary Bayesian interpretation of the target distribution originally described by Liu et al. (2017) breaks down in several places. The issues we found motivate seeking an alternative interpretation that more accurately reflects the structure of the target distribution. In this section, we first show that $q(\theta)$ can be viewed as a Boltzmann distribution before arguing that the framework of general Bayesian inference (Bissiri et al., 2016) provides a much more theoretically sound Bayesian interpretation.

6.3.1 Boltzmann/Gibbs Distribution

An alternative interpretation of the target distribution (4.5) is that it resembles a Boltzmann distribution (Wikipedia contributors, 2026*a*) more than a Bayesian posterior. The Boltzmann or Gibbs distribution satisfies

$$p_i \propto \exp(-\frac{\epsilon_i}{k_B T}),$$

where ϵ_i is the energy at state i and $k_B T$ is the product of the Boltzmann constant and thermodynamic temperature. We then rewrite this as $q(\theta) \propto \exp(-\beta E(\theta))$, for an energy function $E(\theta)$ where β is the inverse of the temperature. Taking logs gives us

$$\log q(\theta) \propto -\beta E(\theta).$$

From (4.5) we have $\log q(\theta) = \frac{1}{\alpha}J(\theta) + \log q_0(\theta) - \log Z$ giving us an energy function $E(\theta) = -J(\theta) - \alpha \log q_0(\theta)$ and the target "posterior" distribution in Boltzmann form

$$q(\theta) \propto \exp(-\beta E(\theta)),$$

where the inverse temperature $\beta = \frac{1}{\alpha}$. This gives more meaning to the definition of α as the temperature. However, it cannot be interpreted as a temperature in the true sense of the Boltzmann distribution since, unless we remove the prior distribution, the energy function depends on α. We again have the temperature parameter playing two distinct roles, here changing both the scaling via β as well as altering the shape of the energy distribution. Under this interpretation, policies with larger expected returns are assigned lower energy and therefore greater probability mass, while the prior distribution acts as a scalable penalty in the energy function.

While this interpretation does not require treating $\exp(\frac{1}{\alpha}J(\theta))$ as a likelihood, the energy function being dependent on the temperature weakens a literal thermodynamic analogy. Because of this limitation, we introduce a final, more principled, Bayesian interpretation.

6.3.2 General Bayesian Inference

General Bayesian inference (Bissiri et al., 2016) is a framework for updating a prior belief distribution to a posterior when the parameter of interest is connected to observations via a loss function $l(\theta, x)$. In this case, the general Bayesian posterior is defined

$$p(\theta|x) \propto p_0(\theta) \exp\left(-l(\theta, x)\right).$$

From this generalised view, ordinary Bayesian belief updating is recovered in the case that the loss function is equal to the negative log-likelihood function.

It is immediately apparent that this formulation provides an appealing interpretation for the SVPG target distribution because our earlier critiques showed that $\exp(\frac{1}{\alpha}J(\theta))$ does not behave as a true likelihood. Instead, negative expected returns are naturally interpreted as a loss function as in our original equation (3.1). From this perspective, the SVPG target distribution becomes

$$q(\theta) \propto \exp\left(-l(\theta)\right) q_0(\theta)$$

where the loss function $l(\theta) = -\frac{1}{\alpha}J(\theta)$. This interpretation bypasses the need to construct a fixed likelihood function, and is directly relevant to reinforcement learning, where the primary objective is defined as maximisation of an objective function (or minimisation of a loss function) rather than as a probability distribution for a fixed dataset. Recently, Kato (2026) applied this formulation to policy learning due to the difficulty of obtaining a true likelihood function in RL, which supports the idea of using general Bayesian inference in RL.

This formulation does not remove the practical issues mentioned in Section 6.2.1. However, it provides a much more consistent interpretation of the SVPG target distribution than an ordinary Bayesian posterior.

Chapter 7

Conclusion

Our primary aim for this dissertation was to investigate whether Stein Variational Policy Gradient (SVPG) can be interpreted as a variational inference method for updating a set of policy parameters to approximate a Bayesian posterior. Chapter 4 motivated the need for such a method by outlining the lack of uncertainty representation that algorithms which compute point estimates provide. This chapter also broadly described the ways we could use such a Bayesian posterior distribution to quantify uncertainty and make informed decisions, particularly in high-risk situations. By studying policy gradient estimators, Bayesian inference, and SVPG itself, this dissertation has argued that while SVPG performs well as an optimisation method for encouraging diversity in a set of updating policy particles, its interpretation as approximating a Bayesian posterior is weak for a number of reasons.

The experiments in chapter 5 showed that, even when we scale α to increase the magnitude of the policy gradient, prior distributions have a much stronger and more persistent effect on learning than we expect in ordinary Bayesian inference. Using a prior distribution as a form of regularisation was shown to be weak in our experiment, with only a very low temperature allowing regularised particles to learn substantially. The diagnostic comparisons between the policy and prior gradient magnitudes demonstrated that the influence of the regularising prior did not reduce as training progressed.

This behaviour differs from ordinary Bayesian inference, where the effect of the prior is gradually overpowered as more data is observed. In SVPG, however, the policy gradient does scale with data. And in fact, the precise notion of data in the SVPG target distribution is not well defined. This suggests that the quantity used in SVPG is not functioning like the gradient of a genuine likelihood in the usual Bayesian sense.

However, the dissertation also shows that the persistence of the prior can be helpful when encoding informative prior information. When the prior distribution was centred around parameters already known to be located in a region of high expected return, learning was improved dramatically. The persistence of the prior gradient resulted in more stable learning than that of the particles initialised at the high performing θ without a prior term.

In conclusion, the results of this dissertation suggest that SVPG is better understood as an optimisation algorithm than as a variational inference method

approximating a Bayesian posterior. The ordinary Bayesian interpretation is weakened by the lack of a true fixed likelihood, the persistent influence of the prior, and the finite-particle and policy gradient approximations. We argued that a more convincing interpretation is therefore to interpret SVPG as a general Bayesian inference method, using a more natural loss function rather than a likelihood.

References

ApX Machine Learning (2026), 'Generalized advantage estimation (gae) — advanced rl'. Online resource.

Arora, S. and Doshi, P. (2021), 'A survey of inverse reinforcement learning: Challenges, methods and progress', *Artificial Intelligence* **297**, 103500.

Bellman, R. (1954), 'The theory of dynamic programming', *Bulletin of the American Mathematical Society* **60**(6), 503–515.

Bissiri, P. G., Holmes, C. C. and Walker, S. G. (2016), 'A general framework for updating belief distributions', *Journal of the Royal Statistical Society Series B: Statistical Methodology* **78**(5), 1103–1130.

Blei, D. M., Kucukelbir, A. and McAuliffe, J. D. (2017), 'Variational inference: A review for statisticians', *Journal of the American statistical Association* **112**(518), 859–877.

Bottou, L. (2010), Large-scale machine learning with stochastic gradient descent, *in* 'Proceedings of COMPSTAT'2010: 19th International Conference on Computational StatisticsParis France, August 22-27, 2010 Keynote, Invited and Contributed Papers', Springer, pp. 177–186.

Bowling, M., Martin, J. D., Abel, D. and Dabney, W. (2023), Settling the reward hypothesis, *in* 'International Conference on Machine Learning', PMLR, pp. 3003–3020.

Chandak, S., Shah, P., Borkar, V. S. and Dodhia, P. (2024), 'Reinforcement learning in non-markovian environments', *Systems & Control Letters* **185**, 105751.

Chung, W., Thomas, V., Machado, M. C. and Le Roux, N. (2021), Beyond variance reduction: Understanding the true impact of baselines on policy optimization, *in* 'International conference on machine learning', PMLR, pp. 1999–2009.

Das, A. and Nagaraj, D. (2023), 'Provably fast finite particle variants of svgd via virtual particle stochastic approximation', *Advances in Neural Information Processing Systems* **36**, 49748–49760.

Denardo, E. V. (1967), 'Contraction mappings in the theory underlying dynamic programming', *Siam Review* **9**(2), 165–177.

Djerbi, R., Rouane, A., Taleb, Z. and Saradouni, S. (2025), 'Design and implementation of a self-driving car using deep reinforcement learning: A comprehensive study', *Computers & Industrial Engineering* **207**, 111319.

Doya, K. (1996), 'Efficient nonlinear control with actor-tutor architecture', *Advances in neural information processing systems* **9**.

Farama Foundation (2024*a*), 'CartPole environment — Gymnasium classic control'.
URL: *https://gymnasium.farama.org/environments/classic_control/cart_pole/*

Farama Foundation (2024*b*), 'Lunar Lander environment — Gymnasium box2d'.
URL: *https://gymnasium.farama.org/environments/box2d/lunar_lander/*

Farama Foundation (2025*a*), 'FrozenLake environment — Gymnasium toy text'.
URL: *https://gymnasium.farama.org/environments/toy_text/frozen_lake/*

Farama Foundation (2025*b*), 'MountainCar environment — Gymnasium classic control'.
URL: *https://gymnasium.farama.org/environments/classic_control/mountain_car/*

Gabry, J., Simpson, D., Vehtari, A., Betancourt, M. and Gelman, A. (2019), 'Visualization in bayesian workflow', *Journal of the Royal Statistical Society Series A: Statistics in Society* **182**(2), 389–402.

Gaon, M. and Brafman, R. (2020), Reinforcement learning with non-markovian rewards, *in* 'Proceedings of the AAAI conference on artificial intelligence', Vol. 34, pp. 3980–3987.

Ghavamzadeh, M., Engel, Y. and Valko, M. (2016), 'Bayesian policy gradient and actor-critic algorithms', *Journal of Machine Learning Research* **17**(66), 1–53.

Ghavamzadeh, M., Mannor, S., Pineau, J. and Tamar, A. (2015), 'Bayesian reinforcement learning: A survey', *Foundations and Trends® in Machine Learning* **8**(5-6), 359–483.

Greensmith, E., Bartlett, P. L. and Baxter, J. (2004), 'Variance reduction techniques for gradient estimates in reinforcement learning', *Journal of Machine Learning Research* **5**(Nov), 1471–1530.

Grondman, I., Busoniu, L., Lopes, G. A. and Babuska, R. (2012), 'A survey of actor-critic reinforcement learning: Standard and natural policy gradients', *IEEE Transactions on Systems, Man, and Cybernetics, part C (applications and reviews)* **42**(6), 1291–1307.

Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018), Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, *in* 'International conference on machine learning', Pmlr.

Havrilla, A., Du, Y., Raparthy, S. C., Nalmpantis, C., Dwivedi-Yu, J., Zhuravinskyi, M., Hambro, E., Sukhbaatar, S. and Raileanu, R. (2024), 'Teaching large language models to reason with reinforcement learning', *arXiv preprint arXiv:2403.04642* .

Huang, Y., Cao, R. and Rahmani, A. (2022), Reinforcement learning for sepsis treatment: A continuous action space solution, *in* 'Machine Learning for Healthcare Conference', PMLR, pp. 631–647.

Kang, P., Tobler, P. N. and Dayan, P. (2024), 'Bayesian reinforcement learning: A basic overview', *Neurobiology of Learning and Memory* .
URL: *https://www.sciencedirect.com/science/article/pii/S1074742724000352*

Kato, M. (2026), 'General bayesian policy learning'.
URL: *https://arxiv.org/abs/2602.23672*

Kingma, D. P. and Ba, J. (2014), 'Adam: A method for stochastic optimization', *arXiv preprint arXiv:1412.6980* .

kjytay (2018), 'Bayesian interpretation of ridge regression'.
URL: *https://statisticaloddsandends.wordpress.com/2018/12/29/bayesian-interpretation-of-ridge-regression/*

Kober, J., Bagnell, J. A. and Peters, J. (2013), 'Reinforcement learning in robotics: A survey', *The International Journal of Robotics Research* **32**(11), 1238–1274.

Konda, V. and Tsitsiklis, J. (1999), 'Actor-critic algorithms', *Advances in neural information processing systems* **12**.

Kumar, V. (2020), 'Mathematical analysis of reinforcement learning — bellman optimality equation'. Medium / TDS Archive.
URL: *https://medium.com/data-science/mathematical-analysis-of-reinforcement-learning-bellman-equation-ac9f0954e19f*

Lawrence, G., Cowan, N. and Russell, S. (2012), 'Efficient gradient estimation for motor control learning', *arXiv preprint arXiv:1212.2475* .

Lee, G., Hou, B., Mandalika, A., Lee, J., Choudhury, S. and Srinivasa, S. S. (2018), 'Bayesian policy optimization for model uncertainty', *arXiv preprint arXiv:1810.01014* .

Lehmann, M. (2024), 'The definitive guide to policy gradients in deep reinforcement learning: Theory, algorithms and implementations', *arXiv preprint arXiv:2401.13662* .

Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. and Wierstra, D. (2015), 'Continuous control with deep reinforcement learning', *arXiv preprint arXiv:1509.02971* .

Littman, M. L. (1994), Markov games as a framework for multi-agent reinforcement learning, *in* 'Machine learning proceedings 1994', Elsevier, pp. 157–163.

Littman, M. L. (2009), 'A tutorial on partially observable markov decision processes', *Journal of Mathematical Psychology* **53**(3), 119–125.

Liu, Q. (2017), 'Stein variational gradient descent as gradient flow', *Advances in neural information processing systems* **30**.

Liu, Q. and Wang, D. (2016), 'Stein variational gradient descent: A general purpose bayesian inference algorithm', *Advances in neural information processing systems* **29**.

Liu, Y., Ramachandran, P., Liu, Q. and Peng, J. (2017), 'Stein variational policy gradient', *arXiv preprint arXiv:1704.02399* .

Mei, J., Chung, W., Thomas, V., Dai, B., Szepesvari, C. and Schuurmans, D. (2022), 'The role of baselines in policy gradient optimization', *Advances in Neural Information Processing Systems* **35**, 17818–17830.

Melcher, M., Weissmann, S., Wilson, A. C. and Zech, J. (2025), 'Adaptive kernel selection for stein variational gradient descent', *arXiv preprint arXiv:2510.02067* .

Mnih, V. (2013), 'Playing atari with deep reinforcement learning'.
URL: *https://arxiv.org/pdf/1312.5602*

Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D. and Kavukcuoglu, K. (2016), Asynchronous methods for deep reinforcement learning, *in* 'International conference on machine learning', PmLR, pp. 1928–1937.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G. et al. (2015), 'Human-level control through deep reinforcement learning', *nature* **518**(7540), 529–533.

Nachum, O., Norouzi, M., Xu, K. and Schuurmans, D. (2017), 'Bridging the gap between value and policy based reinforcement learning', *Advances in neural information processing systems* **30**.

Nemati, S., Ghassemi, M. M. and Clifford, G. D. (2016), Optimal medication dosing from suboptimal clinical examples: A deep reinforcement learning approach, *in* '2016 38th annual international conference of the IEEE engineering in medicine and biology society (EMBC)', IEEE, pp. 2978–2981.

Ng, A. Y., Russell, S. et al. (2000), Algorithms for inverse reinforcement learning., *in* 'Icml', Vol. 1, p. 2.

Peters, J. (2010), 'Policy gradient methods', *Scholarpedia* **5**(11), 3698.

Puterman, M. L. (1990), 'Markov decision processes', *Handbooks in operations research and management science* **2**, 331–434.

Raghu, A., Komorowski, M., Ahmed, I., Celi, L., Szolovits, P. and Ghassemi, M. (2017), 'Deep reinforcement learning for sepsis treatment', *arXiv preprint arXiv:1711.09602* .

Salim, A., Sun, L. and Richtarik, P. (2022), A convergence theory for svgd in the population limit under talagrand's inequality t1, *in* 'International Conference on Machine Learning', PMLR, pp. 19139–19152.

Schulman, J., Chen, X. and Abbeel, P. (2017), 'Equivalence between policy gradients and soft q-learning', *arXiv preprint arXiv:1704.06440* .

Schulman, J., Levine, S., Abbeel, P., Jordan, M. and Moritz, P. (2015), Trust region policy optimization, *in* 'International conference on machine learning', PMLR, pp. 1889–1897.

Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P. (2015), 'High-dimensional continuous control using generalized advantage estimation', *arXiv preprint arXiv:1506.02438* .

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017), 'Proximal policy optimization algorithms', *arXiv preprint arXiv:1707.06347* .

Sell, T. and Singh, S. S. (2023), 'Trace-class gaussian priors for bayesian learning of neural networks with mcmc', *Journal of the Royal Statistical Society Series B: Statistical Methodology* **85**(1).

Shakya, A. K., Pillai, G. and Chakrabarty, S. (2023), 'Reinforcement learning algorithms: A brief survey', *Expert Systems with Applications* **231**, 120495.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T. et al. (2017), 'Mastering chess and shogi by self-play with a general reinforcement learning algorithm', *arXiv preprint arXiv:1712.01815* .

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D. and Riedmiller, M. (2014), Deterministic policy gradient algorithms, *in* 'Proceedings of the 31st International Conference on Machine Learning', Vol. 32 of *Proceedings of Machine Learning Research*, PMLR, pp. 387–395.

Stan Development Team (2024), 'Posterior and prior predictive checks'. Stan User's Guide, accessed 12 March 2026.
URL: *https://mc-stan.org/docs/stan-users-guide/posterior-predictive-checks.html*

Sumiea, E. H., Abdulkadir, S. J., Alhussian, H. S., Al-Selwi, S. M., Alqushaibi, A., Ragab, M. G. and Fati, S. M. (2024), 'Deep deterministic policy gradient algorithm: A systematic review', *Heliyon* **10**(9).

Sutton, R. S. and Barto, A. G. (2018), *Reinforcement Learning: An Introduction*, 2nd edn, MIT Press.

Sutton, R. S., McAllester, D., Singh, S. and Mansour, Y. (1999), 'Policy gradient methods for reinforcement learning with function approximation', *Advances in neural information processing systems* **12**.

Szita, I. and Lörincz, A. (2006), 'Learning tetris using the noisy cross-entropy method', *Neural computation* **18**(12), 2936–2941.

Whitehead, S. D. and Lin, L.-J. (1995), 'Reinforcement learning of non-markov decision processes', *Artificial intelligence* **73**(1-2), 271–306.

Wikipedia contributors (2026*a*), 'Boltzmann distribution'.
URL: *https://en.wikipedia.org/wiki/Boltzmann_distribution*

Wikipedia contributors (2026*b*), 'Likelihood function'.
URL: *https://en.wikipedia.org/wiki/Likelihood_function*

Williams, R. J. (1992), 'Simple statistical gradient-following algorithms for connectionist reinforcement learning', *Machine learning* **8**(3), 229–256.

Yamaguchi, S., Naoki, H., Ikeda, M., Tsukada, Y., Nakano, S., Mori, I. and Ishii, S. (2018), 'Identification of animal behavioral strategies by inverse reinforcement learning', *PLoS computational biology* **14**(5), e1006122.